EEG-Driven Spiking Neural Network Virtual Keyboard in Software and Hardware Environments

Final Report

BE 4999 – Winter 2026

April 21, 2026

Prepared by:

Chloe Orlandi, Madalyn Fox, Kailey MacFadyen, Michael Josh Pasamba,

Ava Waligora, Mary Fayad, & Nicklas Hanson

Abstract [All]

The designed project is an EEG-driven virtual keyboard intended to aid those with impaired motor abilities and improve technology in non-verbal communication. The system integrates a brain–computer interface (BCI) to interpret electroencephalogram (EEG) signals from a database in order to find the associated character. To achieve a low energy, low latency design, a spiking neural network (SNN) classifier was implemented with field programmable gate array (FPGA) acceleration. An EEG dataset was preprocessed, reduced in feature space, and converted into spike trains. A three layer fully connected SNN was trained to classify signal presence. Character selection was done by calculating the probabilities of P300 signal presence per row/column and finding the intersection with the highest score. The trained SNN was implemented onto an Artix7 FPGA. Final character selection accuracy was calculated as 48.31 ± 22.15% on CPU and 4.76% on FPGA. CPU accuracy was significantly higher than the random weight classifier baseline ($p \ll 0.05$). The system had a total latency of 137.0 ± 30.9 ms for full pipeline on CPU and 113.2 ± 30.9 ms using FPGA acceleration. Energy consumption for the virtual keyboard was 109.21 μJ on CPU and 69.98 μJ on FPGA. A comparison with a traditional artificial neural network (ANN) with similar accuracy revealed that the SNN is lower energy but higher latency. The acceleration of BCI systems using FPGA shows promise for creating a more affordable and accessible device.

Table of Contents

1 Introduction....1
1.1 History....1
1.2 P300 Signal....2
1.3 Classification....3
2 Materials & Methods....4
2.1 Professional Standards....4
2.2 Materials....5
2.2.1 FPGA Board & Power Connection....6
2.2.2 Computer & Software....7
2.2.3 EEG Database....8
2.3 Methods....9
2.3.1 Preprocessing....9
2.3.2 Feature Extraction....12
2.3.3 Spike Encoding....13
2.3.4 SNN Architecture....14
2.3.5 SNN Training....16
2.3.6 Character Selection....18
2.3.7 FPGA Implementation....23
2.4 Validation Tests....24
2.4.1 Preprocessing....24
2.4.2 SNN Binary Classification....25
2.4.3 Character Selection....26
2.4.4 FPGA....27
2.4.5 Energy & Latency....28
2.4.5.1 Pipeline Performance....28
2.4.5.2 ANN Baseline Comparison Model....32
3 Results....34
3.1 Final Design....34
3.1.1 Final Training & Architecture....34
3.1.2 End to End Pipeline/Interactable....39
3.2 Validation Results....42
3.2.1 Preprocessing....42
3.2.2 SNN Binary Classification....43
3.2.3 Character Selection....46
3.2.4 FPGA....48
3.2.5 Energy & Latency....50
3.2.5.1 Finalized ANN Comparison Model....50

3.2.5.2 Pipeline Performance....50
3.2.5.3 ANN/SNN Performance Comparison....54
4 Discussion....56
4.1 Project Successes & Key Achievements....56
4.2 Problem Solving & Design Implementation....57
4.2.1 Preprocessing & Feature Extraction....57
4.2.2 Spike Encoding....58
4.2.3 SNN Architecture & Training....58
4.2.4 Character Selection....59
4.2.5 FPGA Implementation....60
4.2.6 Energy & Latency....60
4.3 Challenges & Limitations....61
4.4 Future Work & Potential Improvements....64
5 Conclusion....65
References....68
Appendix....75

1 Introduction

1.1 History

While considered a newer technology, brain-computer interfaces (BCIs) have experienced rapid development in recent years and are being used in various clinical, engineering, and other applications. These functions include improving communication of the disabled or impaired through the use of measuring electrical signals in the brain [1]. An EEG-driven virtual keyboard's purpose is to display the correct letter based on an electroencephalogram (EEG) signal, achieved through the use of a BCI. These BCI virtual keyboards present a non-invasive solution to an ongoing concern.There are numerous real-world applications and existing research focused on enhancing virtual communication technology. BCIs have advanced in the last fifty years from a concept that only seemed achievable in science fiction, to now being possible in reality as it continues to develop [2]. Applications focused on BCIs have already positively impacted impaired patients and are becoming more common in clinical diagnosis. Currently, researchers continue to seek ways to make the process quicker and less costly in order to improve communication and better understand how the human brain functions. In recent Neuralink studies, Skyler Granatir comments on ALS that it interferes with communication between the brain and muscles, comparing it to a severed internet table, but BCIs can bypass this [3]. Yan explains that communication is possible through an EEG by measuring their output electrical signals, and successfully analyzing them through specific commands [4]. In both academic research and real-world applications, this process is designed to be completely safe and non-invasive as signals are measured through electrodes placed on the scalp [5].

1.2 P300 Signal

Event related potentials (ERPs) produced in the brain are commonly used in virtual keyboard applications, most notably the P300 signal [6], [7]. The P300 signal can link brain activity to letters and is reported to be reliable and repeatable [8]. To stimulate a P300 response, the individual being monitored must recognize that a task-related rare event has occurred [9]. This response has been used for virtual spelling by randomly intensifying rows or columns in a matrix of characters, which evokes time dependent P300 responses corresponding to the position in the matrix of the desired character to be typed [6], [8]. The P300 signal is characterized by a positive potential approximately 300ms after the stimulus event (*Figure 1)*.

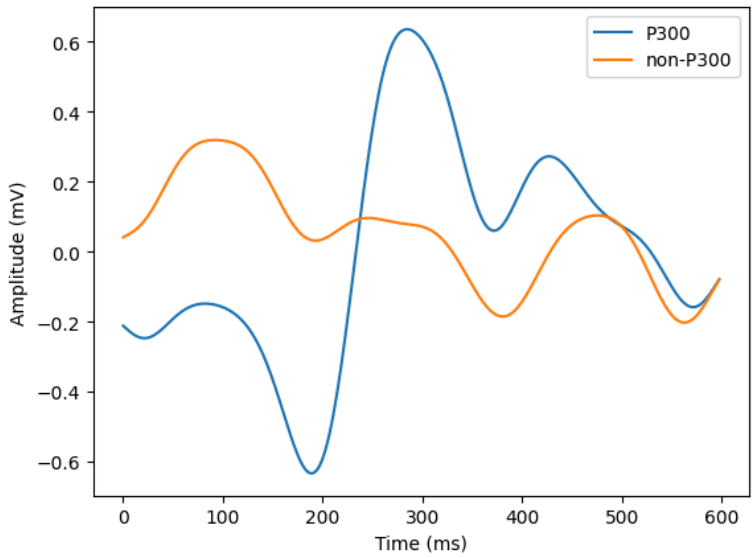


Figure 1: *Grand Averaged midline response (Pz, Fz and Cz electrodes) from the training set across all subjects after a stimulus event. The P300 signal shows a relatively positive spike in amplitude around 300ms and a relatively negative amplitude compared to the non-P300 signal*

1.3 Classification

A classification algorithm is used to interpret these signals and produce the correct character. With the rise of artificial intelligence (AI) in everyday use, it is able to be integrated with existing assistive technologies [10]. Spiking neural networks (SNNs) are a developing field in AI, and are becoming more prevalent in different AI applications due to their close resemblance to the human brain [1]. They have shown similar performance to traditional artificial neural networks (ANNs) at a much lower energy cost [11]. This makes them a promising resource in EEG software development.

Hardware has been used in addition with SNNs in the form of Field-Programmable Gate Array (FPGA) chips. A key function of hardware in spiking neural networks is reducing logic operation in a device [12]. This allows for higher efficiency and lower power consumption. Additionally, parallelism allows for low latency, higher performance, and less data processing time [13]. These aspects of FPGAs allow them to be much faster than Central Processing Units (CPUs), while having lower energy consumption than Graphics Processing Units (GPUs) [14]. FPGAs are especially useful in SNNs due to their flexible memory, digital signal processing (DSP) capabilities, and usefulness for mapping spike encoding/decoding [15].

Many preexisting EEG based BCI systems rely on traditional learning approaches such as recurrent neural networks (RNNs) for signal classification. While various models like this can have strong accuracy, they often have a fixed window of operation and batch-based processing. This can cause latency issues for the system and delay in the virtual keyboard [16]. This delay heavily affects the user's experience and fails to meet real-time requirements such as a low

latency response time. In contrast, SNNs process information in an event-driven sequence and temporally precise manner. This aligns more closely with the structure of the EEG signals and allows for easier classification decisions. In addition, utilizing a FPGA for processing this signal can reduce latency and power consumption. As a result, an SNN-based approach with FPGA acceleration may offer more suitable tradeoffs between latency, efficiency, power consumption, and classification performance for embedding an EEG driven virtual keyboard.

A virtual keyboard application using SNN for classification of a P300 speller paradigm was designed and underwent various testing and learning cycles. The team put a heavy emphasis on developing software incorporating a SNN architecture. Additionally, hardware was utilized in the form of a FPGA board in order to provide comparison with CPU in terms of latency and accuracy. The design serves as a basis for the future of virtual keyboards that will improve accuracy, reduce latency, and reduce power consumption. These advancements will improve communication not only with speech impaired but open many other technological opportunities.

2 Materials & Methods

2.1 Professional Standards

The development of the EEG-driven virtual keyboard followed established professional and ethical engineering standards, particularly those outlined in the IEEE Code of Ethics, which emphasizes safety, accountability, and transparency in system design [17]. Because this system interprets neural signals for communication, ensuring reliable and low-latency performance was critical to user safety and effectiveness. Design constraints such as maintaining response times under 300 ms and validating classification accuracy were implemented to meet functional safety

requirements. Additionally, the project aligns with expectations for reproducibility by using structured data processing, defined training and testing procedures, and documented system architecture.

Regulatory and ethical considerations were also incorporated, particularly those related to Software as a Medical Device (SaMD) and risk management standards such as ISO 14971 [18], [19]. Potential risks, including signal noise, misclassification, and delays, were addressed through preprocessing, filtering, and validation techniques. The handling of EEG data also requires attention to user privacy and informed consent, as neural signals may contain sensitive information [20]. Overall, the project integrates ethical responsibility, regulatory awareness, and standard engineering practices to ensure a safe, reliable, and responsible assistive technology.

2.2 Materials

The overall cost of the system is summarized in *Table 1*, with a total project cost of $310.57. This low cost is achieved by relying on publicly available EEG data and open-source software tools, eliminating the need for expensive biomedical acquisition hardware while still supporting full system development and evaluation.

Table 1: *Cost Summary*

Item Name	Quantity	Cost	Taxes	Total
Nexys A7 AMD Artix™ 7	1	$290.00	$17.40	$307.40
USB/Micro B Cable	1	$2.99	$0.18	$3.17
Computer*	1	$0.00	$0.00	$0.00
Software Tools**	N/A	$0.00	$0.00	$0.00
EEG Database***	N/A	$0.00	$0.00	$0.00
			Project Total Cost:	**$310.57**

**Used personal laptops*
***Freely available software tools, with exception of MATLAB, which was accessed through Oakland University*
****Used public database*

The materials selected for this project were chosen to support the development, testing, and hardware implementation of an EEG-driven virtual keyboard system while maintaining low cost and accessibility. The design uses commercially available hardware and open-source software tools, along with publicly available data, to reduce overall system cost without sacrificing performance.

2.2.1 FPGA Board & Power Connection

The primary hardware component used is the Nexys A7 AMD Artix™-7 FPGA board [21], which is used to implement the SNN system in hardware. Based on the weighted decision matrix shown in *Table 2*, the Nexys A7 achieved the highest overall score (4.3) when evaluated against key design criteria including I/O ports, RAM, latency, operating voltage, power consumption, and cost. Its superior performance in categories such as RAM and latency, along with strong

overall capability, makes it well-suited for real-time neuromorphic signal processing and system implementation.

Table 2: *Weighted decision matrix evaluating candidate FPGA boards. The Nexys A7 AMD Artix ™ 7 achieves the highest overall score and is selected for the project.*

Design Criteria	Weight	FPGA Model	
		Basys 3 AMD Artix™ 7 FPGA Trainer Board	**Nexys A7 AMD Artix ™ 7**
I/O Ports	3	3	5
RAM	5	3	5
Latency	5	3	5
Operating Voltage	3	5	5
Static Power Consumption	4	4	3
Cost	4	5	3
	Score:	**3.8**	**4.3**

A USB/Micro B cable is used to provide power and enable communication between the FPGA board and a host computer. This connection is required for programming the board and transferring data during development and testing.

2.2.2 Computer & Software

A personal computer is used for all stages of software development, including preprocessing, feature extraction, spike encoding, and model training. The laptop also supports simulation and testing environments, allowing the full pipeline to be validated prior to hardware implementation. Since personal devices were used, no additional cost was incurred.

Several software tools were used to develop and evaluate the system. Python 3.13.5 served as the primary programming environment, with libraries such as NumPy [22] for numerical operations, SciPy [23] for signal processing, and PyTorch [24] with SNNTorch [25] for neural network implementation. MATLAB-compatible data formats were used for dataset handling, and MATLAB R2025a was used for initial preprocessing code and testing. Development environments such as Visual Studio Code and Google Colab were also utilized. Additionally, Xilinx Vivado Design Suite 2025.2 was used for FPGA design, synthesis, and implementation for a hardware-compatible format.

2.2.3 EEG Database

The system utilizes a publicly available EEG dataset, eliminating the need for specialized data acquisition hardware. This significantly reduces project cost while still providing high-quality data for training and evaluation. The database for both training and testing of the virtual keyboard was collected in [8]. The data was obtained from 55 healthy participants, with signals recorded from 32 scalp electrodes positioned according to the international 10-20 system (*Figure 2*). The EEG signals were sampled at 512 Hz, which provides sufficient temporal resolution for capturing the P300 data. During the experiment, characters were arranged in a 6x6 matrix, and rows and columns were intensified in a randomized sequence to evoke a P300 signal, which occurs approximately 300 ms after presentation of a target stimulus. Each stimulus flash lasted 125ms with shorter inter-stimulus intervals of 62.5ms, and 15 repetitions were done for each character epoch to improve the signal-to-noise (SNR) ratio [8].

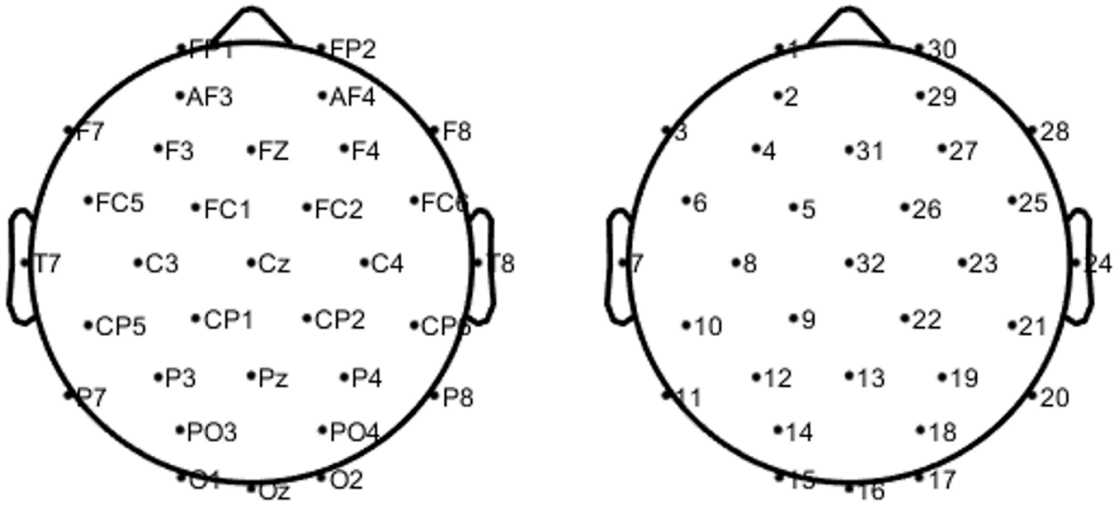


Figure 2: *International 10-20 electrode placement names (left) and numbers (right). From [8]*

The continued EEG recordings were split into labeled character epochs, creating a three dimensional dataset organized by character epoch, time sample, and EEG channel. Each epoch contains information about when a stimulus occurred, which row or column had flashed, whether the flashed object included the target character, and the correct character label. The dataset for each subject has been split into two parts: a training set and a testing set. The training set contains raw EEG readings for the spelling of two words, “BRAIN” and “POWER”, for a total of 10 characters for each subject. The testing set comparatively had subjects spell four words: “SUBJECT”, “NEURONS”, “IMAGINE”, and “QUALITY”.

2.3 Methods

2.3.1 Preprocessing

The preprocessing stage prepares the raw electroencephalography (EEG) signals for feature extraction by removing noise, isolating relevant signal components, and organizing the data into consistent epochs. This stage is important due to the low signal-to-noise ratio of P300 EEG

signals and their susceptibility to artifacts such as eye blinks, muscle activity, and baseline drift [26], [27]. The implemented preprocessing function follows a structure of baseline correction, detrending, filtering, and epoch extraction to ensure compatibility with the overall pipeline.

Raw EEG data are loaded from MATLAB .mat files using the mat73 library. Each dataset contains multiple trials, where each trial includes continuous EEG recordings, stimulus markers, and associated labels. The sampling rate is extracted directly from the dataset, 512 Hz [8]. Based on this sampling rate, two key time windows are defined: a baseline window of 200 ms and an epoch window of 600 ms. This aligns with the temporal characteristics of the P300 response, which typically occurs within 300 ms after stimulus onset [8], [26].

To reduce dimensionality and focus on the most relevant neural activity, a subset of eight EEG channels known to exhibit strong P300 responses is used (*Table 3*). These channels were chosen as they provide classification comparable to a full set of 64 electrodes [28].

Table 3: *Selected EEG channels and corresponding 10–20 system electrode locations.*

Channel Number	Electrode Name
31	Fz
32	Cz
12	P3
13	Pz
19	P4
14	PO3
18	PO4
16	Oz

Epoch extraction is performed by identifying stimulus onset indices using the provided marker arrays. For the training dataset, two types of events are considered: target stimuli and non-target stimuli. For each valid stimulus index, an epoch is extracted beginning at the stimulus onset and extending for 600ms.This ensures that the model focuses on stimulus-related neural activity.

Each extracted epoch undergoes baseline correction by subtracting the mean signal value computed over the pre-stimulus baseline window (0.2 seconds prior to the stimulus). This step removes DC offsets and compensates for slow signal drift. Following this, linear detrending is applied to each epoch to eliminate any residual linear trends in the signal which helps reduce low-frequency artifacts [27], [8].

A fourth-order Butterworth bandpass filter is applied to each epoch to isolate the frequency components most relevant to the P300 response. Two frequency ranges were evaluated during development: 0.5-10 Hz and 0.5-15 Hz. These ranges are commonly used in P300 and event-related potential (ERP) studies, as it preserves the relevant low-frequency components of the signal while attenuating high-frequency noise and slow drift [8]. The filter is implemented using zero-phase forward-backward filtering to avoid phase distortion and preserve the temporal structure of the EEG signals.

For testing and character selection, a similar preprocessing pipeline is applied with slight modifications. Instead of class labels, each epoch is associated with a flash identifier corresponding to the row or column stimulus (values 1–12), which is later used in the character selection algorithm. Additionally, a specialized preprocessing function enables extraction and

processing of a single character at a time, allowing for efficient evaluation and debugging of the system.

The final output of the preprocessing stage is a structured dataset of filtered EEG epochs, along with corresponding labels or flash identifiers and a time vector representing the duration of each epoch. This ensures smooth integration with the overall system pipeline.

2.3.2 Feature Extraction

For efficient training, relevant features must be extracted from the data to reduce noise and amplify the differences between classes. After preprocessing, epochs are downsampled using decimation by average technique, similarly to the one applied in [8]. The goal of downsampling is to increase the SNR by averaging together multiple time samples. The number of time samples averaged together is known as the factor. After downsampling, the 0 to 600ms initial time window will be further narrowed down to 200 to 400ms. As the P300 signal occurs at approximately 300ms after stimulus [9], this time window accounts for variation among subjects and trials while removing fringe noise that may dilute the strength of the signal. Data is standardized per channel using a standard scaler, represented in equation (1), where μ is the mean and s is the standard deviation:

$$Z_{ch} = \frac{x_{ch} - \mu_{ch}}{s_{ch}} \tag{1}$$

Standardizing per channel allows for the relative amplitudes of each channel to be brought to the same range. Standardization removes any bias from certain channels with larger relative

magnitudes overpowering the signal from other channels. This is why standardizing per channel, as opposed to a global standardization, is important.

2.3.3 Spike Encoding

Following preprocessing and feature extraction, the EEG signals were converted into spike-based representations using a delta encoding approach. Spike encoding is required because SNNs operate on discrete spike events rather than continuous-valued signals [29]. In this implementation, delta encoding was selected due to its ability to represent temporal changes in the signal efficiently while reducing redundant spike activity compared to traditional rate-based methods [30]. While rate-based encoding maps signal magnitude to firing frequency, delta encoding reduces overall spike density and computational load, which is beneficial for both software simulation and FPGA implementation [31]. Rather than encoding absolute signal magnitude, delta encoding generates spikes based on changes between consecutive time steps. Additionally, this method avoids the need for precise timing calibration required in temporal encoding schemes, while still preserving important dynamic information in the EEG signal.

The implemented encoding method computes the difference between consecutive samples for each channel and trial, and generates a spike when the change exceeds a defined threshold. Positive changes greater than or equal to the threshold produce a spike of +1, while negative changes below the negative threshold produce a spike of −1 when off-spike encoding is enabled. This allows the model to capture both increases and decreases in signal amplitude, improving the temporal representation of the EEG features. A threshold value of 0.09 was selected based on empirical testing to balance sensitivity to meaningful signal changes while minimizing

noise-induced spikes. The resulting spike tensor is structured as (trials, time, channels), aligning with the input requirements of the SNN model and integration with the existing pipeline. The selected delta encoding approach provides an efficient and hardware-compatible method for transforming continuous EEG features into spike trains suitable for SNN-based classification.

2.3.4 SNN Architecture

The base unit of the SNN is the leaky integrate and fire (LIF) neuron, which acts as a simple model of a biological neuron. The LIF neuron has previously been used in BCI system applications, including P300 signal classification [32], with high accuracy, and offers a low complexity, biologically plausible design [11]. LIF neurons are commonly used in FPGA implementations of SNNs due to their balance of simplicity and accuracy [33], [34], [35], allowing for low power systems with minimal hardware resource usage compared to more complex neuron models. Because of these aspects, LIF neurons can be used in resource, cost, and power constrained projects.

The LIF neuron operates through updating an inner membrane potential whenever a binary spike (1) is read as an input. The effect this spike has on membrane potential depends on the weight of the connection ('integrate'). Whenever the membrane potential reaches a certain threshold value, it will output a binary spike (1), and the membrane potential will be reset (the 'fire'). The membrane potential will decay over time without another stimulus (the 'leak').

We used first-order LIF neurons in our SNN implementation. The design is modeled using discrete time steps (t) for use in a digital neural network and is represented by the following

equation derived in [25], where U is the membrane potential, β is the decay constant, W is a learnable weight, and X[t + 1] is the input spike at the next time step:

$$U[t + 1] = \beta U[t] + WX[t + 1] - S[t]U_{\text{thr}} \tag{2}$$

S[t] stores the binary output spike of the neuron, which will read 1 when $U(t) > U_{thr}$, and 0 otherwise. U_{thr} is the threshold membrane potential. When a spike occurs, the membrane potential is reset by subtracting U_{thr} from the membrane potential [25].

The LIF neuron can be represented visually in *Figure 3*. Each input spike causes a sharp jump in membrane potential, followed by a time-step mediated potential decay. When the neuron has been spiked enough, membrane potential will reach the threshold and the neuron will output a spike. More frequent and/or higher weighted input spikes will therefore reach the threshold more often, and produce more output spikes, leading to a stronger signal.

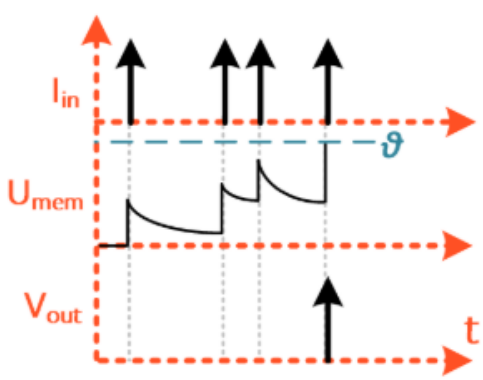


Figure 3: *LIF neuron response graph over time. I_{in} represents the input spikes to the neuron, U_{mem} the membrane potential, and V_{out} the output spikes of the neuron, and t is time. ϑ is the threshold potential. Adapted from [25].*

The LIF neurons are combined into a fully connected network. Fully connected layers were chosen due to their relative simplicity and ease of transfer to FPGA hardware. Each connection consists of a weight and bias that determines the effect of the input neuron on the output neuron. The weights of the SNN before training were initialized using a Kaiming uniform distribution [36], and the biases were initialized in a uniform distribution from -0.1 to 0.1.

2.3.5 SNN Training

Supervised training of the SNN made use of surrogate gradient descent and backpropagation through time (BPTT). Because spikes are non-differentiable, a surrogate gradient must be used during backpropagation to allow for learning [25]. For this purpose, we use Equation (3), the fast sigmoid gradient implemented in [25], to represent the relationship between S and U, where S is the output spike, U is the membrane potential, and k is a tunable parameter.

$$\frac{\partial \hat{S}}{\partial U} \leftarrow \frac{1}{(1+k|U|)^2} \tag{3}$$

BPTT calculates the loss gradient through all time steps, allowing for past time steps to influence the gradient [25]. BPTT is calculated by way of Equation (4) as it was described in [25], where $\mathcal{L}$ is the loss, W is the learnable weights, and s ≤ t are all time steps up to, and including, the current time step. $\frac{\partial \mathcal{L}[t]}{\partial W[s]}$ is calculated through the chain rule during the backwards pass.

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t} \sum_{s \leq t} \frac{\partial \mathcal{L}[t]}{\partial W[s]} \tag{4}$$

Weighted cross entropy loss will be the loss metric for SNN training. Loss is calculated based on the spike rate of the two output classes. Spike rate is simply the sum of spikes across all time steps divided by the total time:

$$SR = \frac{\sum_t S[t]}{t} \tag{5}$$

Softmax probability of each class out of a total of C classes based on spike rate will be calculated by way of Equation (6), adapted from a similar equation in [25] which uses membrane potential instead.

$$p_i = \frac{e^{SR_i}}{\sum_{i=1}^{C} e^{SR_i}} \tag{6}$$

Weighted cross entropy loss finds the loss resulting from comparing the true output y with the output probability p from softmax, weighted by a weight w. This loss is described in Equation (7), adapted from similar equations in [24] and [25]:

$$\mathcal{L}_{CE} = -\sum_{i=1}^{C} w_i y_i log(p_i) \tag{7}$$

The SNN is trained using the Adaptive Moment Estimation (Adam) optimizer. Adam is robust and shows good performance on recurrent networks [25]. The detailed algorithm for Adam is found in [37]. In short, Adam uses first and second moments of the gradient to compute

individual adaptive learning rates for each parameter, and requires minimal tuning of hyperparameters. The optimizer will adjust the learnable parameters of the SNN during training according to the loss gradient. The standard parameters of Adam were used during training.

The learning rate for the SNN was initially set to 5×10^{-4}. A learning rate scheduler was applied during the training that reduces the learning rate by a factor of 0.5 upon a plateau in the validation loss. A patience of 10 epochs was set before the reduction factor was applied. The SNN was trained on batched training data input. Dropout for each hidden layer in the SNN was applied to reduce overtraining and increase robustness. A random seed of 42 was selected to ensure reproducible results during training.

2.3.6 Character Selection

All decision activity for character selection is driven by the initial thirty-six character matrix shown to the subject and generating a positive response. The subject would receive a series of high speed flashes on a display. If the subject's letter is flashed on the display, a reaction should occur around the 300 millisecond mark after the flash is shown. In the experiment, each stim sequence was blinked 15 times to print a single character. The target characters blinked a total of 30 times, while the non-target characters blinked a total of 150 times [8]. The delay or refresh period allows the P300 peak to get lower and also helps delineate each individual flash. The timeframe cannot be too slow otherwise the subject’s attention starts to wane.

Figure 4a is the entire matrix shown for reference with *Figure 4-d* being examples of individual rows or columns flashed for the subject and in theory, should only evoke a positive P300

response if the character is present in the individual flash. Flashing rows and columns as opposed to individual characters produces faster and more balanced results. In the character selection process, only twelve flashes (six rows and six columns) are required instead of thirty-six individual character flashes.

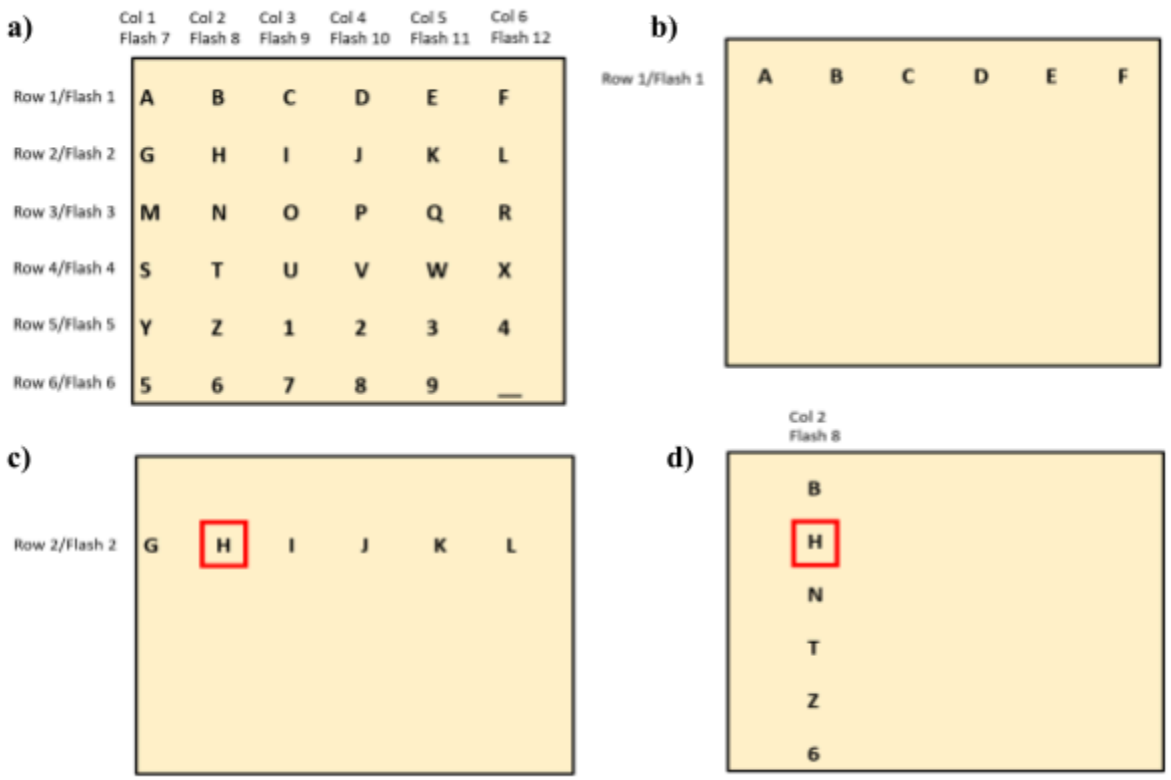


Figure 4: a) *The full matrix of all thirty six characters shown and flashed in the process of data collection in character selection.* ***b)*** *Example of the first flash, displaying only the first row in the flash matrix.* ***c)*** *Example of a sequential row flash, in this case being the second one, with focus on the letter 'H' being the target.* ***d)*** *Example of a sequential row column, in this case being the eighth one, with focus on the letter 'H' being the target.*

Each one of the one-hundred and eighty individual flashes completes the feature extraction and spike encoding. The output for a single flash is shown below in *Figure 5*, depicting fifteen out of 34 total binary data points for either positive P300 or negative P300 reaction. This output is the beginning step of the character selection process.

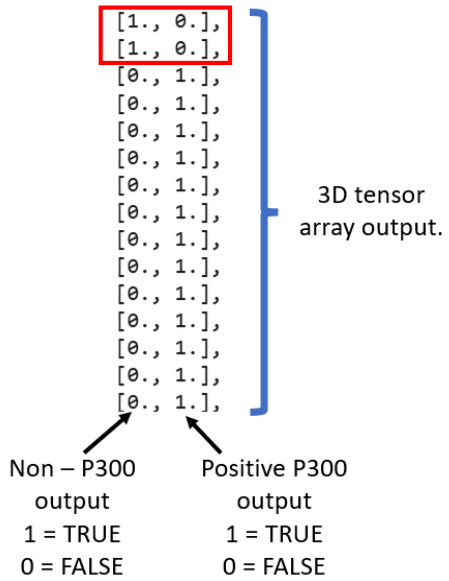


Figure 5: *An example of an output 3D tensor array to be interpreted in character selection. The left column represents Non-P300 hits, while the right column represents Positive P300 hits, both indicated by binary responses.*

There are four potential responses per timestep:

- (0,0); Response is not a non-P300 positive (0); Response is not a P300 positive (0).
- (1,0); Response is a non-P300 positive (1); Response is not a P300 positive (0).
- (0,1); Response is not a non-P300 positive (0); Response is a P300 positive (1)
- (1,1) Response is a non-P300 positive (1); Response is a P300 positive (1)

The character selection section is the process of choosing the correct letter to display. It will implement the selection of 36 alpha-numeric characters. Character selection will be done through the spatial positioning of the target character in the spelling application of the P300 paradigm [38]. The equations altered from [38] listed below demonstrate how the sum of probabilities is calculated through matrices.

$$C_{(j)} = \sum_{i=1}^{n} ln(P^{1}_{(i,j)}) \tag{8}$$

$$index_{row} = argmax\, C_{(j)} \qquad 1 \leq j \leq 6 \tag{9}$$

$$index_{col} = argmax\, C_{(j)} \qquad 7 \leq j \leq 12 \tag{10}$$

Equation (8) sums the natural logarithm of probabilities of having produced a P300 signal from intensifying row or column *j* over all repetitions of the character, where $C_{(j)}$ denotes the sum of probabilities. $P^{1}_{(i,j)}$ is calculated using Equation (6) to get the probability of P300 signal occurring based on spike rate. The second and third equation deal with the matrix row and column intensifications respectively, with *j* in the rows having an index from 1-6 and *j* in the columns having an index from 7-12. Together, these determine the spatial positioning of the target. The implementation in [8] follows a similar process. The program is not choosing a letter directly: it is matching the highest probability in the column with the highest probability in the row based on the P300 speller [8]. *Figure 6* below displays the process of how probability is calculated.

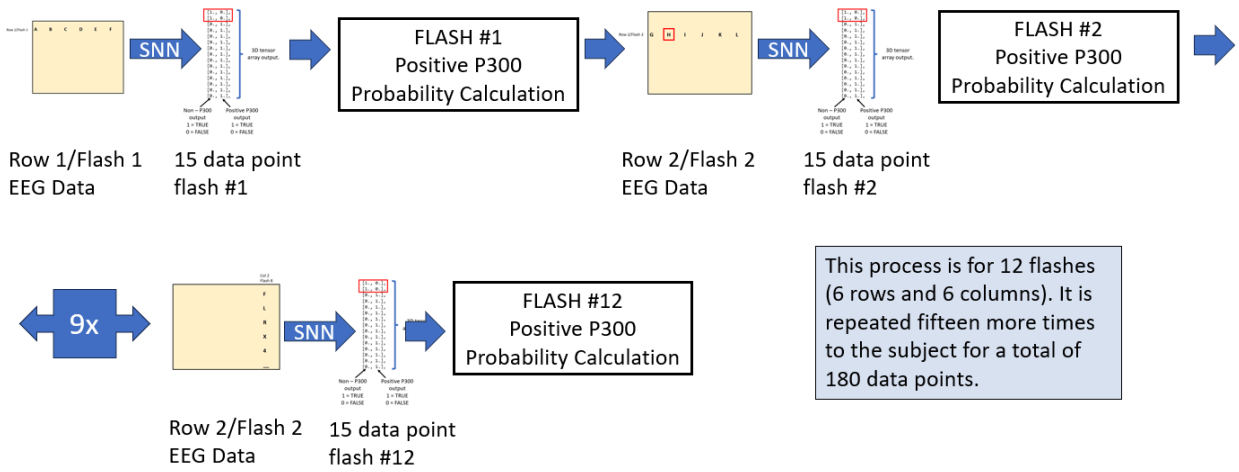


Figure 6: *A chart showing the process of calculating the probability flash matrix. Flashes are cycled through, calculating the probability and collection of tensor data points across repetitions.*

Further breakdown of the character selection process is explained below.

- Positive P300 Probability calculated for flash
- Process is repeated one-hundred and eighty times (12 flashes * 15 repetitions).
- The total probability for each individual row and column is summed.
- Algorithm indexes the highest probability row.
- Algorithm indexes the highest probability column.
- Letter selection (using selected row and column intersection)

For each individual EEG signal representing a single flash of a row or column, the SNN produces a spiking output array with a length of the number of time steps for both P300 and non-P300 classification. With the flash repetitions, values corresponding to each position in the flash matrix are established. Then, with summation, the row and column index positioning is

used to find the highest probability and output the matching character. This summation is repeated for each of the one-hundred and eighty flashes captured (the six rows and the six columns individually flashed and then repeated fifteen times). Additionally, epochs must be reordered in order for the data to properly be interpreted, which is achieved through the use of a corresponding flash ID array.

2.3.7 FPGA Implementation

The SNN was implemented on FPGA using VHDL with a synchronous, clock-driven architecture. At each clock cycle, the read-only memory (ROM) outputs a vector of input spikes which are propagated through the three-layer fully connected network. Spike inputs were represented as signed values [-1, 0, 1], which can decrease, maintain, or increase the accumulation value, respectively. Neuron behavior was implemented using a discrete time accumulation model suitable for digital hardware. At each timestep, input spike values were multiplied by their corresponding synaptic weights that were learned from the CPU model, and accumulated to update the membrane potential of the neuron.

Trained network weights were stored in on chip memory and initialized using coefficient (.coe) files. A fixed point signed representation was used for weights and intermediate values to ensure efficient hardware implementation. During operation, weights were accessed based on neuron index and input position, allowing reuse of memory resources across timesteps. To manage the execution of the network, a finite state machine (FSM) was used to control the data progression through each layer and across timesteps. The FSM ensured that computations occur in proper sequence, layer activation and synchronization were coordinated with the system clock. Input

spike data was provided through ROM and outputs were observed through simulation for verification purposes.

The membrane potential was compared to a predefined threshold at each timestep. If the threshold was exceeded, the neuron generated an output spike; otherwise, there was no spike produced. Following a spike event, the membrane potential was reduced to model a reset condition. The membrane potential and intermediate values were stored in registers, allowing the neuron state to stay consistent across clock cycles. This enables temporal accumulation of spike activity over multiple time steps. The outputs from each layer were then propagated forward as inputs to the following layer within the same timestep. This process was repeated for each timestep, allowing spike information to move through the network sequentially.

2.4 Validation Tests

2.4.1 Preprocessing

During the development of the fourth-order Butterworth bandpass filter, two frequency ranges were evaluated: 0.5-10 Hz and 0.5-15 Hz. To assess their effectiveness, raw EEG signals were plotted alongside the corresponding filtered outputs for each frequency range. These visual comparisons were used as a validation method to evaluate noise reduction, signal smoothness, and preservation of key waveform features. By analyzing how closely the filtered signals maintained the overall shape and prominent peaks of the raw signal while minimizing noise, an appropriate frequency range for the preprocessing pipeline was selected.

2.4.2 SNN Binary Classification

The preallocated training set of the dataset was split further into a training, validation, and test set of sizes 70%, 15%, and 15% of the original, respectively. In total, the training set contained 69300 P300 or non-P300 samples while the validation and test sets contained 14850 samples. The validation set was used alongside the training set to monitor for overtraining. The test set was used as a final evaluation to confirm that no biases were present in the validation set.

To ensure minimum overtraining of the model and as a measure of training progress, cross-entropy loss was logged across epochs for both the training and validation sets, as well as computed during evaluation of the test set. The loss is reduced across each batch of data by taking the weighted mean of the individual loss with equation (11), adapted from [24]. The subsequent batch means are averaged to give an overall score for each dataset.

$$\mathcal{L}_{batch} = \sum_{n=1}^{N} \frac{\mathcal{L}_{CE}}{\sum_{n=1}^{N} w_{y_n}} \tag{11}$$

Accuracy is often used to determine the success of classification using neural networks. However, accuracy can be unreliable in the case of extremely imbalanced datasets like the one in [Won et al]. Therefore, in addition to classic accuracy, the main metric to determine the level of success of the binary classification training is balanced accuracy. Balanced accuracy represents SNN classification success regardless of the imbalance in the dataset, and is calculated using equation (12) from [39], where TP is true positives, FN is false negatives, TN is true negatives,

and FP is false positives. All accuracy metrics were computed on the training set, validation set, and testing set for binary classification.

$$balanced\ accuracy = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right) \tag{12}$$

Additionally, the spike rate of the output and average maximum membrane potential were monitored in the validation set to determine whether the SNN is exhibiting proper spiking behavior and is not saturated or suppressed. The average spike rate of the validation set was calculated through averaging the spike rate of each batch, while the average membrane maximum was the average of the maximum membrane potential recorded for each batch.

2.4.3 Character Selection

In order to evaluate and test the character selection, the preallocated 'test' data from the dataset was split 25% into a validation set and 75% into the final testing set. This resulted in a validation size of 385 characters (7 characters per subject) and a test set size of 1155 (21 characters per subject). Character selection accuracy using a random weight binary classifier was calculated on the test set as a baseline for random classification. Additionally, accuracy for each individual subject was separately evaluated. This separation can indicate whether some subjects perform better on our classifier than others.

To determine the overall success of character selection, accuracy across the whole validation or testing set was determined through a mean of subject accuracies. This metric from the validation set was used to optimize the character selection algorithm while the test set accuracy is an

unbiased reflection of the virtual keyboard's success. In addition to overall accuracy, the standard deviation based on individual subject accuracy is calculated to more fully capture the variation between subjects when reporting a single value. Sample standard deviation is calculated with one degree of freedom using the following formula [22]:

$$s = \sqrt{\frac{\sum_i \left|a_i - \bar{a}\right|^2}{N-1}} \tag{13}$$

In the case of character selection, N is the number of subjects, a_i is the accuracy of the current subject, and $\bar{a}$ is the mean of all subject accuracies.

To determine whether the trained classifier performs significantly better than random, a two-sided paired t-test was done comparing the random weight subject accuracies on the test set with their trained classifier counterparts. A significant difference was declared if $p \leq 0.05$.

2.4.4 FPGA

The FPGA implementation was validated through a combination of simulation and on-chip debugging using an Integrated Logic Analyzer (ILA). Waveform analysis, along with manual calculation verification, confirmed that neuron membrane potentials accumulated correctly at each timestep and that spike outputs were generated only when the threshold condition was met. Membrane values were properly reset between input samples, ensuring no leakage values carried over between sequences. A primary focus of the validation process was ensuring that neuron states were updated exactly once per timestep. Improper timing such as early or delayed updates

would lead to incorrect accumulation, inflated membrane values, and ruin the remaining portions of sequence.

Layer-by-layer validation confirmed that spike outputs from the first layer propagated to the following layers with no loss of data. In addition, timing analysis confirmed that each layer completed its computation within the expected number of clock cycles, and that control signals (start, done, busy and idle) remained properly synchronized throughout the pipeline. To quantitatively evaluate correctness, FPGA spike outputs were compared to spike outputs received via CPU in comparison on the same input sequences.

2.4.5 Energy & Latency

2.4.5.1 Pipeline Performance

A critical metric to understand, and optimize, for the virtual keyboard is the latency. Latency is the elapsed time for a process to execute from beginning to end. For the virtual keyboard to be useful, a character must be able to be selected quickly upon EEG data input otherwise it is impractical for patients. For timely keyboard applications, an internal system latency of under 300 ms is expected. Latency was measured using a monotonic clock of the highest resolution in order to accurately record the elapsed time. Five repetitions of the pipeline were done to determine variance in latency. Because the array size extracted for all characters and all subjects are the same, the latency of a single character was used as an estimate. In order to determine the latency of the entire pipeline, the latency was first recorded for each individual section: preprocessing, feature extraction, delta encoding, SNN module, and character selection. To determine the latency of each section, a timer must be placed that records the start time at the

beginning of the section, and end time once the section has reached the end. This is then done for the five sections of the pipeline, with the total latency being the addition of all five latencies. By knowing how long each section takes to complete, diagnosing where the greatest amount of delay occurs in the pipeline is possible. This data allows for corrections and adjustments to be made to reduce the latency and fulfill the 300 millisecond requirement.

Latency of the SNN on the FPGA was measured via simulation of the process to produce the binary classifications necessary for one character prediction. This simulation measures the clock cycles required and latency in milliseconds can be estimated based on the frequency of the clock.

To estimate how much energy the virtual keyboard will use to select a character, the total floating point operations (FLOPs) of each section were recorded. FLOPs include basic math operations on floating point elements within the system, such as addition, subtraction, multiplication, and division. Since each of these operations will require a small amount of energy on the real hardware, the total amount of FLOPs will give an estimate on how much energy the system uses. Calculating the number of FLOPs for the virtual keyboard will require each part of the pipeline including preprocessing, feature extraction, delta encoding, SNN inference, and character selection. Once the number of FLOPs has been counted per subsection, they are then summed to give the total number of FLOPs for the whole system. The total FLOPS as well as that of each section are then converted into energy using a FLOP to energy conversion factor of 5.6 pJ/FLOP, which will give an approximate energy value [40]. This method will compare the efficiency of different parts of the system without depending on a specific hardware setup.

A breakdown of the FLOP calculations required for each section is included below:

During preprocessing, the EEG data is filtered to include only the selected channels and is then segmented into epochs around each of the stimulus events. For each valid segment, a baseline is found by taking the total mean over a short window, and the number of FLOPs is then counted based on each epoch's mean calculation. The baseline is then subtracted from the signal, followed by detrending to remove the drift, with the FLOPs then counted for each epoch for each operation. Next, the bandpass filter is then applied using a forward and backward filtering process, which contributes to a large number of FLOPs due to its complexity. All FLOPs from each step are then added up to produce the total FLOPs of the preprocessing.

During the feature extraction, the number of FLOPs required for each step was counted similarly to the preprocessing. The data is first downsampled using an averaging method, and the FLOPs are counted based on the number of elements within the averaging method and each average. A time window of interest is then selected from each epoch which will add minimal computational cost. Then the data is normalized per channel by computing the mean and standard deviation and the scaling of the data. The FLOPs are counted for subtraction and division operations during scaling as well as those within the mean and standard deviation operations themselves. All the FLOPs are then added up to produce the total number of FLOPs of the feature extraction.

For the delta encoding, the number of FLOPs is based on the number of elements processed in each epoch. Each data point in the signal requires a fixed number of operations to compare changes against the threshold and determine whether or not a spike occurs. When using a

schematic including off-spikes, three FLOPs per element was estimated to account for comparisons and updates. The total number of FLOPs for each epoch are then computed by multiplying the number of elements by the per-element cost. These values are then summed up across all epochs to obtain the total number of FLOPs for the delta encoding.

To measure the computational cost of the SNN, the number of synaptic operations(SOPs) are used, as they better represent the spike based computation compared to FLOPs [41]. SOPs are calculated by counting the number of spikes and then multiplying it by the number of connections to the next layer. Specifically, input spikes and spikes that are generated in each hidden layer are summed and propagated through the network to estimate the total number of operations. SNNs also incur a small cost for neuron state updates across all the time steps [11]. The total cost of the SNN is the sum of all spiked based and memory related operations, which gives a more accurate estimate of the SNN energy consumption.

Finally, the FLOP count for the character selection is based on the total number of elements processed in the output tensor from the SNN. Each element contributes a fixed number of operations, which accounts for steps such as, averaging, summation, comparison, and selection. It is approximated by assigning a constant number of FLOPs per element and multiplying it by the total number of elements in each tensor. This provides a simplified estimate of computations required to aggregate flash responses and determine the final predicted character.

2.4.5.2 ANN Baseline Comparison Model

To evaluate the performance of the proposed spiking neural network (SNN), an artificial neural network (ANN) was implemented as a baseline model. The ANN was not included as part of the project pipeline but was instead used as a comparison for SNN performance. The ANN was trained and tested using the same preprocessed and feature-extracted EEG data as the SNN.

The ANN was implemented as a feedforward neural network for binary classification of EEG epochs. Feedforward neural networks consist of multiple layers of interconnected neurons, where each neuron computes a weighted sum of its inputs followed by a nonlinear activation function [42]. The model consists of an input layer, two hidden layers, and an output layer.

This structure follows a multilayer perceptron (MLP) design, which is one of the most commonly used ANN architectures for classification tasks [43]. The input to the network can be either precomputed feature vectors or multi-dimensional EEG data, in which it is flattened into a one-dimensional vector before being passed through the network. Each hidden layer includes a linear transformation, batch normalization, a rectified linear unit (ReLU) activation function, and dropout regularization. The final output layer produces logits corresponding to the two classes (target and non-target EEG responses). Weight initialization was performed using Kaiming initialization. Overall, the architecture was selected to balance model complexity and computational efficiency while maintaining a structure comparable to the SNN for evaluation purposes.

The ANN was trained using the same dataset, preprocessing and feature extraction steps as the SNN. The dataset was split into training, validation, and testing subsets using a fixed ratio to ensure consistent evaluation across models. During training, the network processes input data in batches and updates its weights to minimize the difference between predicted and true labels. This learning process is based on adjusting connection weights to reduce error, which is a basic principle of artificial neural networks [42]. Optimization is performed using gradient-based methods, where the error is propagated backward through the network to update weights, referred to as backpropagation [43].

To account for class imbalance in the dataset, class weights were computed from the training data and incorporated into the loss function. In addition, an optional weighted sampling strategy was used to balance class representation within each training batch. Model performance during training was monitored using both accuracy and balanced accuracy to provide a more reliable evaluation when class distributions are uneven.

Training progress was evaluated using both training and validation loss, and a learning rate scheduler was used to adjust optimization behavior over time. To ensure a fair comparison with the SNN, a range of hyperparameters were tested during training, including hidden layer sizes, learning rate, and dropout probability. These parameters were tuned to achieve accuracy levels comparable to the SNN, allowing the ANN to serve as a baseline for comparison.

3 Results

3.1 Final Design

3.1.1 Final Training & Architecture

Raw EEG signals from the dataset were separated into epochs each containing a 600ms period post-flash of a row or column (*Figure 6a*). Eight out of thirty-two channels were chosen for processing to increase SNR and reduce processing and memory requirements. The epochs were preprocessed by applying a 0.5-15 Hz bandpass filter, linear detrending, and baseline subtraction (*Figure 6b*). At the conclusion of preprocessing, each sample is an array of (8 channels, 307 time-steps). After preprocessing, the feature extraction function was applied (*Figure 6c*). Feature extraction consists of three elements: time downsampling by a factor of 3, bringing the effective sample rate down to approximately 171 Hz from the original 512 Hz, time window narrowing to a 200 ms window starting 200 ms post-flash until 400 ms post-flash, and per-channel standardization. The resulting array for one sample after feature extraction is (8 channels, 34 time steps).

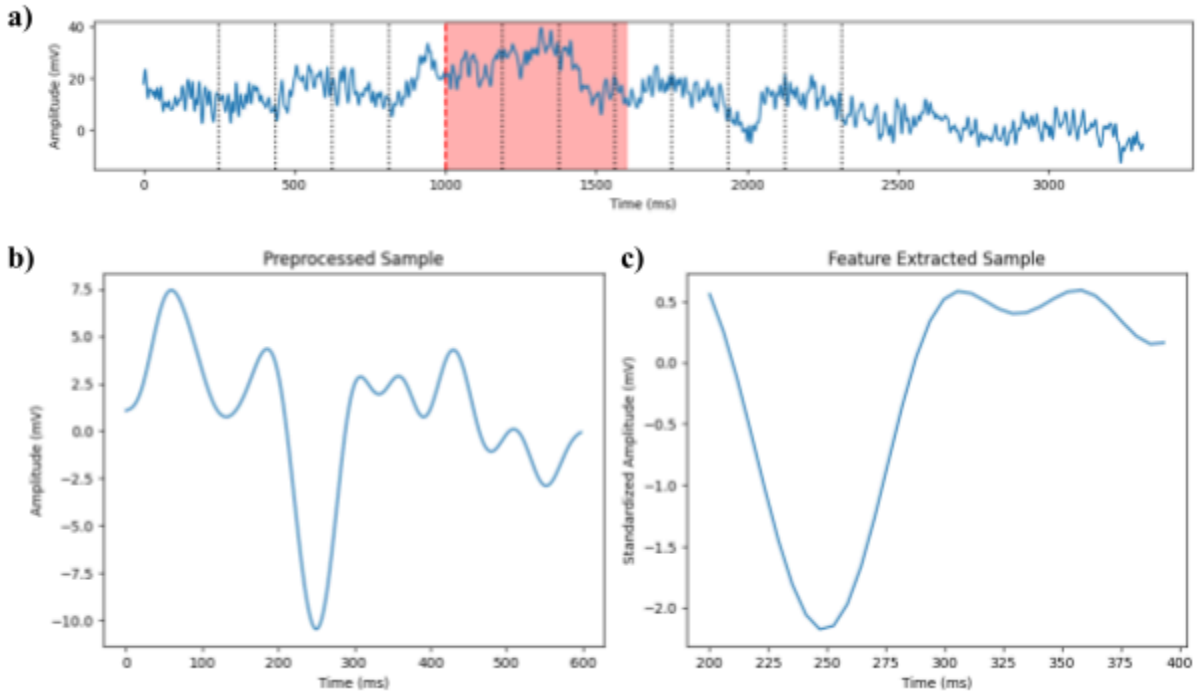


Figure 6: a) *Raw EEG data of one channel for one flash sequence. Each dotted line represents a flash of a row or column. The red shaded area contains a 600 ms extracted sequence after the flash denoted by the red dashed line.* ***b)*** *Shaded 600 ms region after preprocessing.* ***c)*** *Flash epoch after feature extraction*

After data has been processed and relevant features have been extracted, it is encoded into spikes. Delta encoding is applied with a threshold of 0.09 to produce a spike train consisting of only [-1, 0, 1]. An example of delta encoding output for each of the eight channels is shown in *Figure 7*.

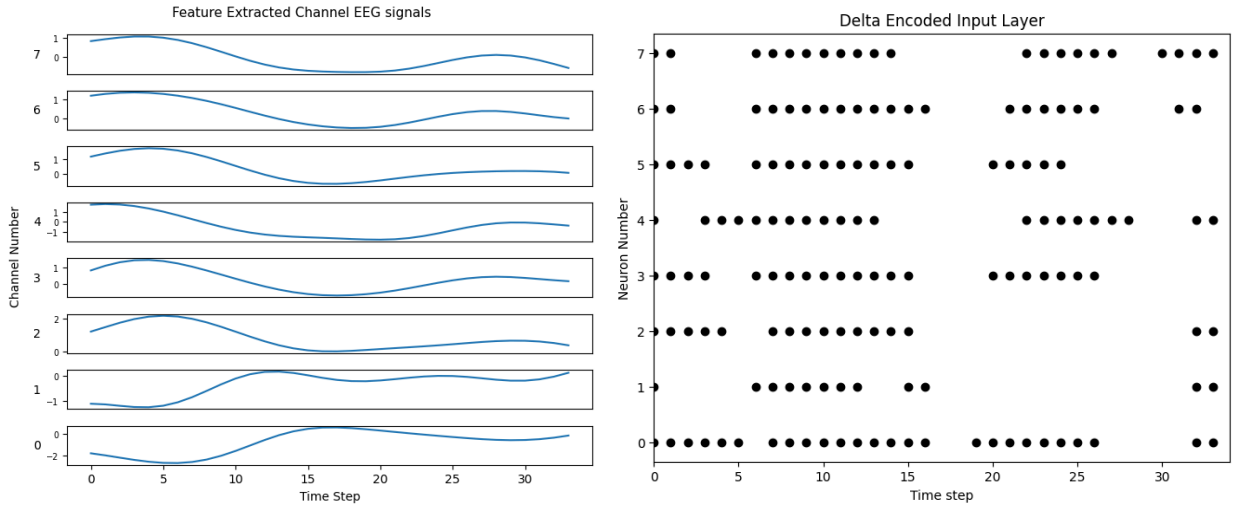


Figure 7: *One sample transformation from feature extraction to delta encoding. The X axis is the time-step used to determine delta encoding by.* ***Left:*** *EEG signals of the 8 channels after feature extraction.* ***Right:*** *Delta-encoded spike trains across input neurons corresponding to the respective channel, showing spikes generated at time steps where the change in signal exceeds the defined threshold of 0.09.*

LIF neurons are organized in a fully connected network with the delta-encoded input, two hidden layers, and an output layer. The specific dimensions of the network are 8 input channels → 128 hidden neurons → 64 hidden neurons → 2 output neurons representing P300 or non-P300 classification (*Figure 8*). This neural network design is small enough to allow for parallel computing of each layer on the FPGA and is simple to implement.

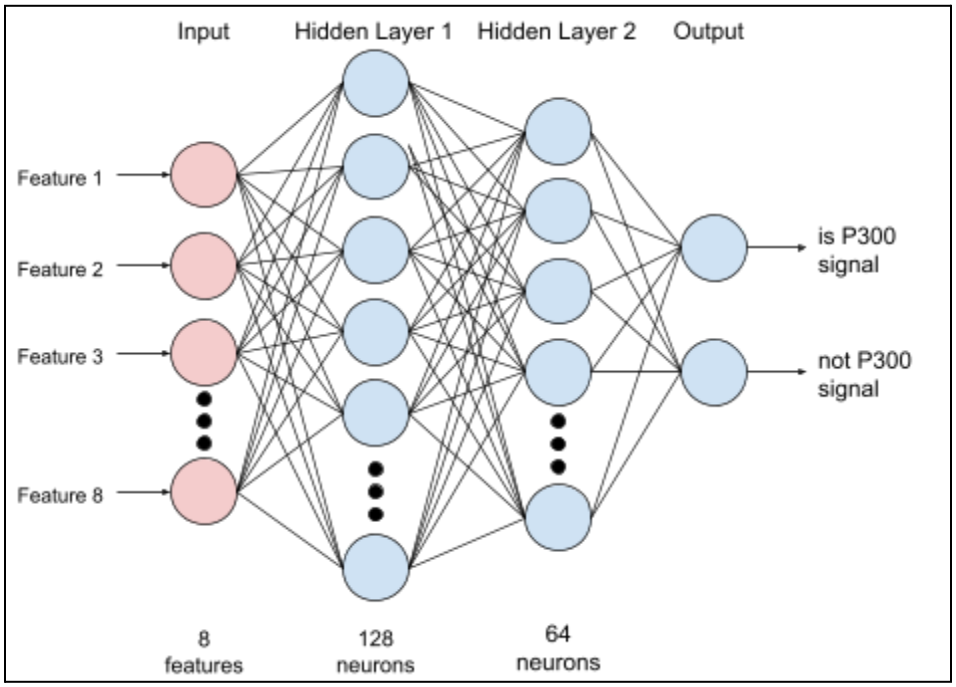


Figure 8: *Spiking neural network architecture. Each blue circle represents an LIF neuron. Each pink circle represents a delta-encoded input. The input layer has 8 channels, hidden layer 1 has 128 neurons, hidden layer 2 has 64 neurons, and the output layer has 2 neurons, which classify the input into whether it contains a P300 signal or not.*

The β and U_{thr} for each of our LIF neurons was chosen to maximize the accuracy during training. From the results of hyperparameter tuning, the final β for all neurons is 0.95, and the U_{thr} of each neuron is 1. The SNN was trained for a total of 100 epochs on the training set with a batch size of 256. For our training, a fast sigmoid constant k=10 was chosen, as it provided the most stable and highest accuracy. A summary of the final design components can be found in *Table 4*.

Table 4: *Final Design Parameter Summary for Pipeline.*

	Final Design Selection	
Preprocessing	Filter Range	0.5-15Hz
Feature Extraction	Downsample Factor	3
	Time Window	200-400ms
Delta Encoding	Threshold	0.09
LIF Neurons	Decay Constant (beta)	0.95
	Threshold	1
SNN Architecture	Hidden Layer Sizes	[128, 64]
	Dropout	0.1
Training	Surrogate Gradient Constant (k)	10
	Batch Size	256
	Epochs	100

Character selection is implemented after the trained SNN is established through the logarithmic probability summation of each row and column. The most probable row and column are used to index for the character in the matrix to be typed.

The final FPGA design implements a fully pipelined SNN architecture in which spike encoded inputs are streamed from an on chip ROM at one timestep per clock cycle, with internal control logic to help coordinate layer synchronization and timestep progression. Each neuron creates parallel fixed-point multiply-accumulate (MAC) operations using preloaded trained weights and biases, making up the majority of the look-up table (LUT) resources. Membrane potentials are updated sequentially with leakage and thresholding, and spike outputs are propagated between

layers through registered signals to help maintain consistency in timing. The design was successfully synthesized and verified through simulation and on chip debugging, showcasing the correct behavior and function of an SNN on a FPGA board.

3.1.2 End to End Pipeline/Interactable

All components of the virtual keyboard were linked together in a pipeline for pseudo-real time selection of a character from a raw EEG signal (*Figure 9*). This pipeline combines preprocessing, extraction of relevant features, SNN binary inference, and selection of the predicted character.

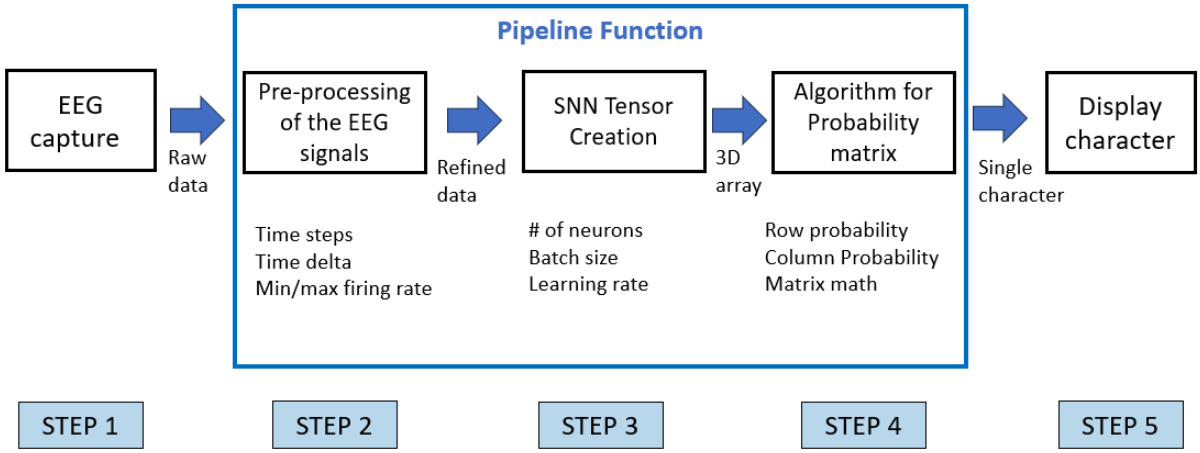


Figure 9: *Flowchart of Overall Process Flow*

Preprocessing, feature extraction, and delta encoding are processed using the same parameters as mentioned previously. The SNN is initiated with the weights and biases that resulted from training. The character selection algorithm is applied after each flash is classified by the SNN to determine the correct row and column of the character.

To demonstrate the capabilities of the virtual keyboard pipeline, a command line application was created for CPU SNN inference. This application produces the predicted character output based on the desired character input for the chosen subject. Upon loading, the user will be prompted to enter the subject number they wish to use (*Figure 10a*). After entering the subject number, the user will be asked to choose a character they wish to type, and the selected output from the pipeline as well as the correct letter will be displayed. If the predicted letter is correct, both the selected output and correct output will be the same (*Figure 10b*). If the predicted letter is wrong, the selected output will display the wrongly chosen character while the correct output will display the originally desired character (*Figure 10c*). The letters typed will be appended the previous outputs to spell out words and other chains of letters (*Figure 10d*). Letters will continue to append to the previous output until the "clear" command is given, which will clear out all previous outputs (*Figure 10e*). To exit the program, the user must type "quit", which will end the session (*Figure 10f*).

a)
```
(.venv) PS C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard> python VirtualKeyboard_cmd_ln_cpu.py
Loading Virtual Keyboard...
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\model_weights\12864_th09_f3_stdf1_weights.pth
Loading Complete
Enter subject number (1-55): |
```

b)
```
Enter subject number (1-55): 7
Enter character to select (A-Z): Y
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\datasets\Won2022_BIDS\.mat_files\s07.mat
Selected output: Y
Correct output: Y
```

c)
```
Enter character to select (A-Z): I
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\datasets\Won2022_BIDS\.mat_files\s07.mat
Selected output: H
Correct output: I
```

d)
```
Enter subject number (1-55): 7
Enter character to select (A-Z): Y
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\datasets\Won2022_BIDS\.mat_files\s07.mat
Selected output: Y
Correct output: Y
Enter character to select (A-Z): E
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\datasets\Won2022_BIDS\.mat_files\s07.mat
Selected output: YE
Correct output: YE
Enter character to select (A-Z): S
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\datasets\Won2022_BIDS\.mat_files\s07.mat
Selected output: YES
Correct output: YES
```

e)
```
Selected output: SENIOR
Correct output: SENIOR
Enter character to select (A-Z): clear
Character Output Cleared
Enter character to select (A-Z): j
C:\Users\crims\Desktop\Senior Design Code\VirtualKeyboard\datasets\Won2022_BIDS\.mat_files\s07.mat
Selected output: J
Correct output: J
```

f)
```
Enter character to select (A-Z): quit
Exiting program...
```

Figure 10: *Command line application for a simulated Virtual Keyboard.* ***a)*** *Subject number input prompt upon loading.* ***b)*** *Letter input that is selected correctly.* ***c)*** *Letter input that is selected incorrectly.* ***d)*** *Chain of multiple letter inputs together.* ***e)*** *Clearing the saved output.* ***f)*** *Exiting the program*

Several wrong inputs are possible when entering subject number or character. If the subject number is not a number but instead a string, the user will be notified of the incorrect format and

prompted again (*Figure 11a*). If the subject number is a number, but not within the range of subjects, the user will be notified that the number is invalid and re-prompt the user for a different number input (*Figure 11b*). If a character is entered that does not exist in those previously recorded in the dataset, the user is notified that the character could not be located and a different character must be entered (*Figure 11c*). These contingencies are placed to prevent the program from crashing upon receiving an invalid input.

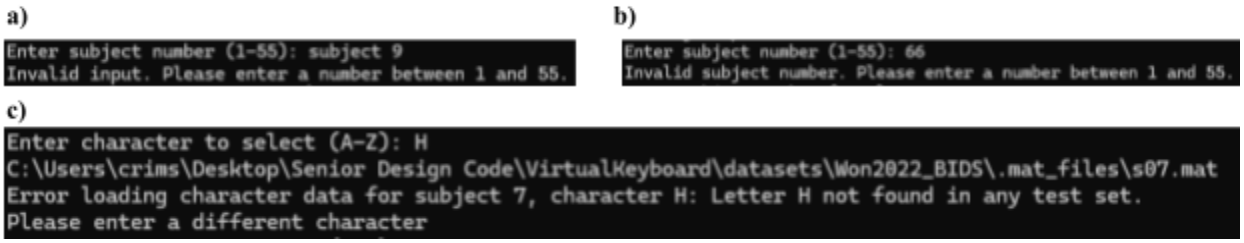


Figure 11: *Incorrect input responses.* ***a)*** *Subject input not a number.* ***b)*** *Subject number does not exist.* ***c)*** *Character does not exist in pre-recorded dataset.*

3.2 Validation Results

3.2.1 Preprocessing

Figure 12 shows a comparison between the raw EEG signal and the bandpass filtered outputs using frequency ranges of 0.5-10 Hz and 0.5-15 Hz, respectively. In both cases, the filtered signals show a reduction in high-frequency noise and overall smoother waveforms compared to the raw signal. The 0.5-10 Hz filter (*Figure 12a*) produces a smoother signal with more attenuation of higher frequencies, which helps reduce noise but also dampens some of the signal peaks. In contrast, the 0.5-15 Hz filter (*Figure 12b*) preserves more of the original signal amplitude and waveform detail. This allows features, such as larger peaks associated with the P300 response, to remain more defined. Based on this comparison, the 0.5-15 Hz filter provides

a better balance between noise reduction and signal preservation, making it more suitable for preserving relevant EEG features needed for classification.

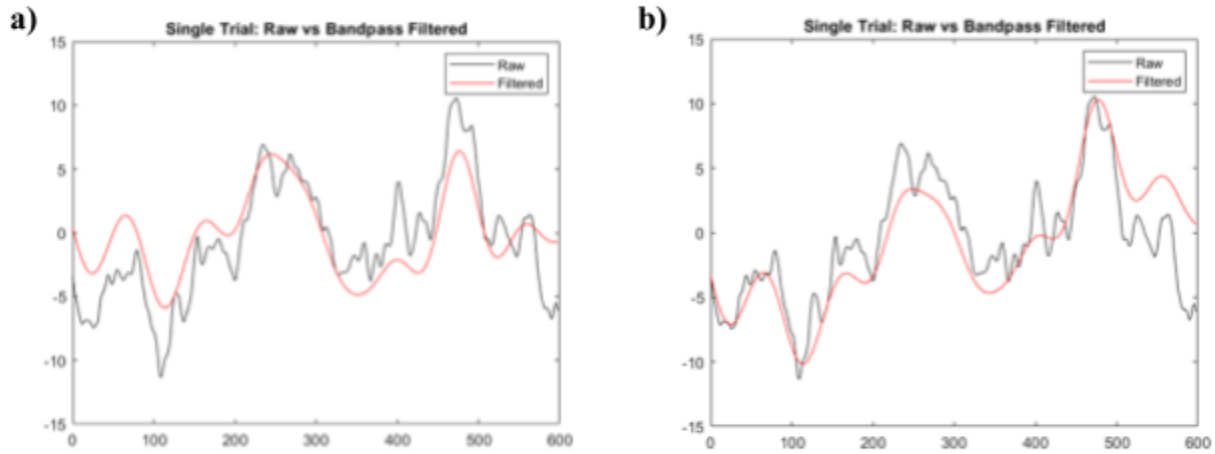


Figure 12: *Comparison of raw EEG signal and bandpass filtered signal:* ***a)*** *using a 0.5–10 Hz frequency range.* ***b)*** *using a 0.5–15 Hz frequency range.*

3.2.2 SNN Binary Classification

The running binary classification loss after being trained for 100 epochs is shown in *Figure 13*. Both the training and validation loss decreased over time; however, validation loss plateaued at around 0.6575. As validation loss plateaued but training loss continued to increase, training was stopped to avoid overtraining on the dataset.

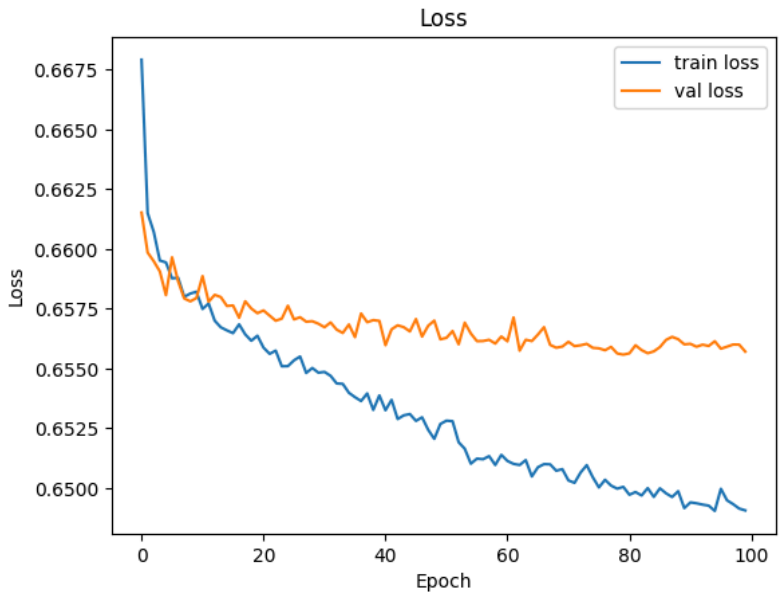


Figure 13: *Training and validation loss of model across training epochs*

Accuracy and balanced accuracy were recorded each epoch in *Figure 14a-b*. Validation accuracy showed extreme variance compared to the training accuracy, while the balanced accuracy is much less variable and stabilized as epochs increased. Balanced validation accuracy showed an increasing trend throughout training but started to plateau after around 50 epochs, while training accuracy had a steady increase.

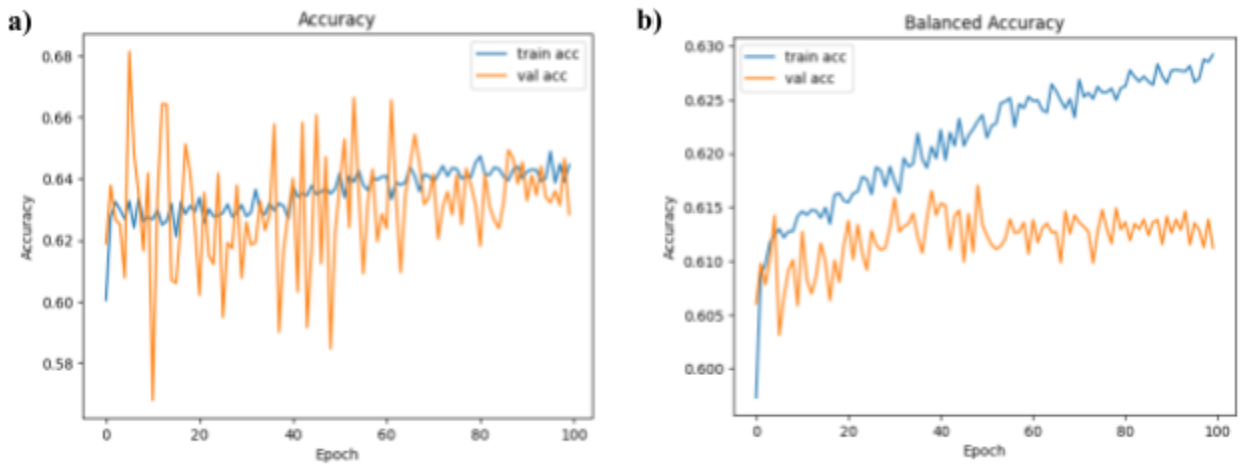


Figure 14: a) *Traditional accuracy of the training and validation sets throughout training.* ***b)*** *Balanced accuracy of the training and validation sets throughout training*

The spiking rate and average maximum membrane potential of the validation set was monitored throughout training as well. At the conclusion of training, the spike rate was 0.4078 and the average maximum membrane potential was 29.6930. The spike rate is healthy for a SNN, indicating that it produces a spike approximately 40% of the time and does not saturate or suppress them. The membrane potential of 29.6930 is high considering our spiking threshold potential is only 1, but as the spike rate is not saturated, this high potential is likely a learned feature to reduce the loss. Any attempts to reduce the membrane potential also reduced accuracy and increased loss.

The final metrics at the conclusion of training are summarized in *Table 5*. The balanced test accuracy was lower than that of both validation and training; however, the difference between testing and validation is only 1.22%. Similar observations can be made of the loss across the datasets as well. The unbalanced accuracy of the test set is much more similar to the validation

set. These results show minimal overtraining or bias, and that our SNN will provide consistent results within the 60% accuracy range.

Table 5: *Final Loss, Accuracy and Balanced Accuracy of SNN after 100 Epochs*

	Loss	**Accuracy**	**Balanced Accuracy**
Training	0.6491	64.45%	62.92%
Validation	0.6557	62.84%	61.12%
Testing	0.6629	62.50%	59.90%

3.2.3 Character Selection

Accuracy on the selected characters was calculated per subject to explore accuracy variability. Both the validation accuracy, which was used as a tool to determine the best method of character selection, and the test accuracy, which is used for final evaluation, was explored (*Figure 15*). Both validation and test accuracy showed extreme variation in accuracy per subject. For the validation set, the accuracy ranged from 0% (0/7) to 100% (7/7), and for the test set, the accuracy ranged from 9.52% (2/21) to 95.24% (20/21).

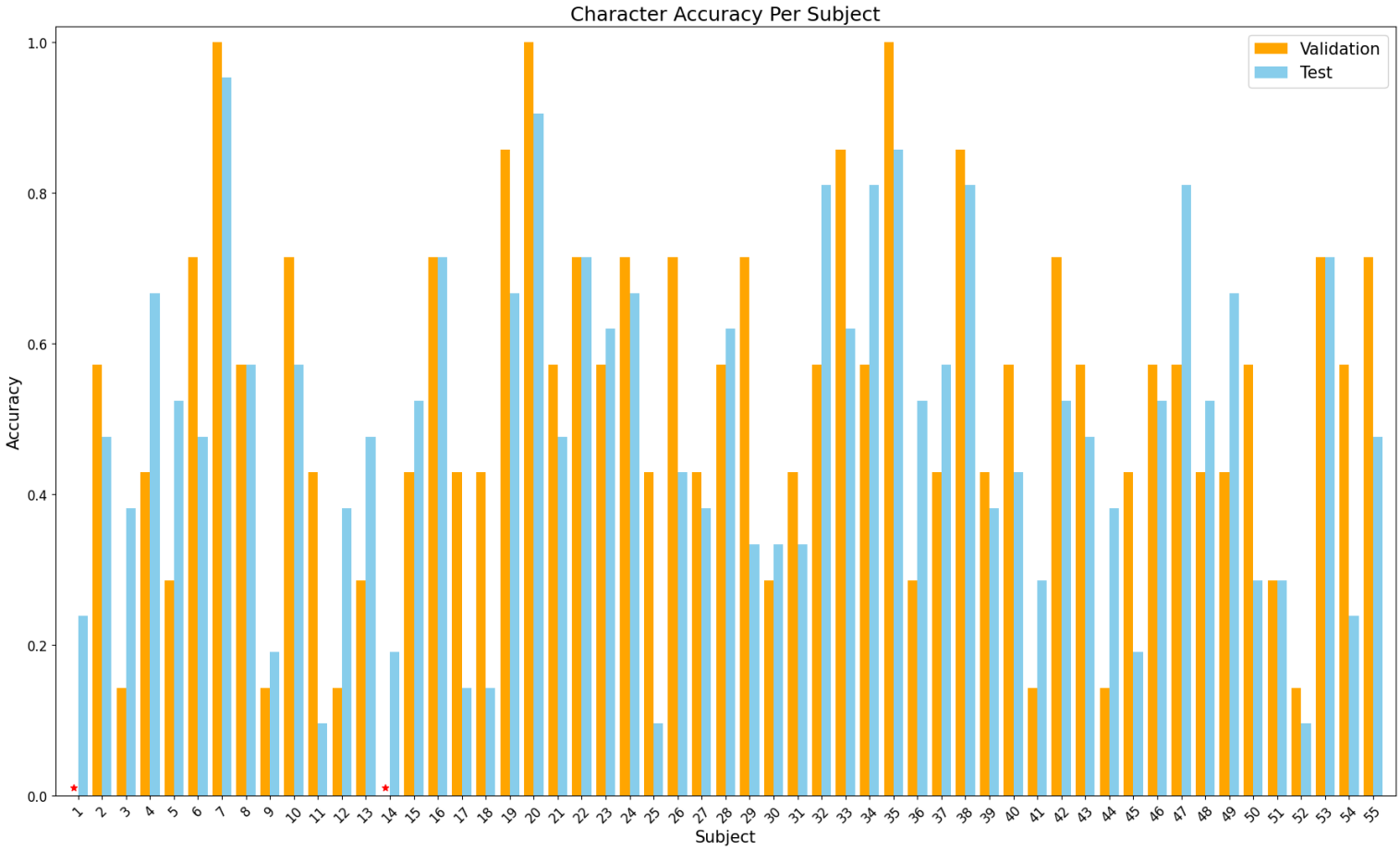


Figure 15: *Per-Subject character selection accuracy for both validation and test sets. The orange bars depict validation accuracy while the blue bars depict test set accuracy. The red stars indicate where the validation accuracy was 0%.*

On a random weight SNN, character selection receives a 9.35 ± 8.69% accuracy. The average accuracy of all subjects during validation and training was 50.91 ± 24.40% and 48.31 ± 22.15% respectively. The test accuracy shows significantly better performance for character selection than the random weight classifier ($p = 8.85e\text{-}20 \ll 0.05$), indicating that the trained SNN can better identify P300 signals than the random weight baseline.

3.2.4 FPGA

The VHDL simulation of the SNN architecture was successful in matching expected outputs among each layer across multiple time steps, proving correct functional behavior. The spike data propagated through all three layers and properly was synchronized between the stages. The layer by layer execution occurred in the correct sequence, confirming the logic was functioning correctly. The design produced stable and consistent intermediate output values, hidden layer 1 with 96% accuracy, hidden layer 2 with 84% accuracy, but output layer 3 had an accuracy of 47%. The performance degraded in the final layer, which indicates a discrepancy between the intermediate computations and the final classification being outputted. This suggests that while the neuron computation and data flow were implemented correctly, there may have been timing alignment errors, weight application, or propagation errors. *Figure 16* displays the process of two neurons using their spikes and weights to produce values within Layer 1. The issue was resolved through control of the enable and state signals, resulting in stable and consistent membrane behavior across all layers.

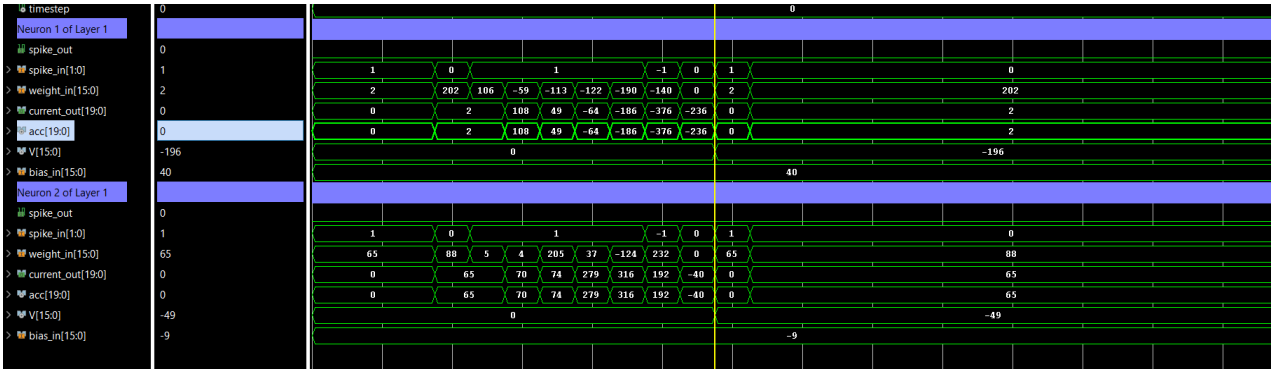


Figure 16: *Simulation analysis showing singular timestep neuron operation in Layer 1, including spike inputs, weight multiplication, accumulation, and membrane potential updates leading to spike generation. Other key components such as bias and weights are displayed.*

Overall, these results show that the SNN was implemented in a functional manner on the hardware, though refinement would be required to improve output accuracy. This reduction in the final layer accuracy is consistent with the overall character selection performance, which gave 4.7% accuracy on the FPGA for a single subject, which is not considered to be above baseline. FPGA resource utilization is dominated by LUTs (44.5%) due to the amount of parallel neurons, with minimal DSP usage (0.42%) due to fixed-point arithmetic, and decent BRAM usage (24.4%) for memory storage (*Figure 17*).

Resource	Utilization	Available	Utilization %
LUT	28210	63400	44.50
FF	7883	126800	6.22
BRAM	33	135	24.44
DSP	1	240	0.42
IO	69	210	32.86

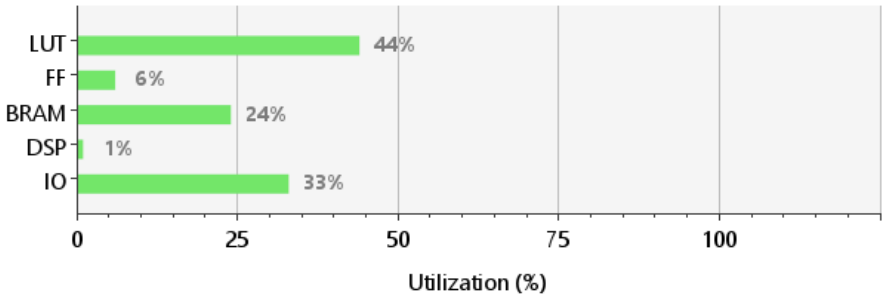


Figure 17: *FPGA resource utilization summary showing usage of LUTs, flip-flops (FF), block ram (BRAM), DSPs, and IO, highlighting a logic-based design with minimal DSP utilization. LUTs at 44% majority due to large amounts of parallel neuron usage.*

3.2.5 Energy & Latency

3.2.5.1 Finalized ANN Comparison Model

To provide a baseline for comparison with the SNN, the final ANN model was evaluated using the same dataset and preprocessing and feature extraction. The ANN was configured as a fully connected multilayer perceptron with two hidden layers of sizes 128 and 64 neurons. A dropout rate of 0.1 was applied after each hidden layer, and ReLU activation functions were used throughout the network. The model was trained using a learning rate of 1×10^{-3} and a weight decay of 5×10^{-4}, with a batch size of 256. A weighted loss function was used to account for class imbalance in the dataset. The model was trained for 100 epochs, with performance monitored at regular intervals. By the final epoch, the model achieved the results shown in *Table 6*.

Table 6: *Accuracy and loss achieved by ANN for the final epoch, validation set, and test set.*

	Accuracy	**Balanced Accuracy**	**Loss**
Final Epoch (100)	73.84%	74.56%	0.5106
Validation Set	69.64%	65.20%	0.6574
Test Set	69.26%	64.32%	0.6724

3.2.5.2 Pipeline Performance

The total latency of the SNN based pipeline on CPU was timed to be 167.0 ± 30.9 ms, combining the individual latency of each pipeline section (*Figure 18*). The majority of the latency is due to preprocessing and SNN inference, with the preprocessing having a latency of 95.2 ± 17.5 ms and the SNN having 67.5 ± 25.3 ms. Contrasting the preprocessing and the SNN,

the feature extraction, delta encoding, and character selection contribute extremely little to the overall latency with the feature extraction having a latency of 3.4 ± 0.7 ms, the delta encoding 0.3 ± 1.0 ms, and the character selection with 0.6 ± 2.2 ms. Overall, these results show that even with the preprocessing and the SNN inference having comparatively large latency, the system is still capable of being used in a virtual keyboard with fast output. The latency of the FPGA SNN inference was estimated to be 13.7 ms through performance metrics on one character's worth of binary inference. The latency for FPGA was lower than the CPU counterpart, proving that FPGA can be used to speed up inference.

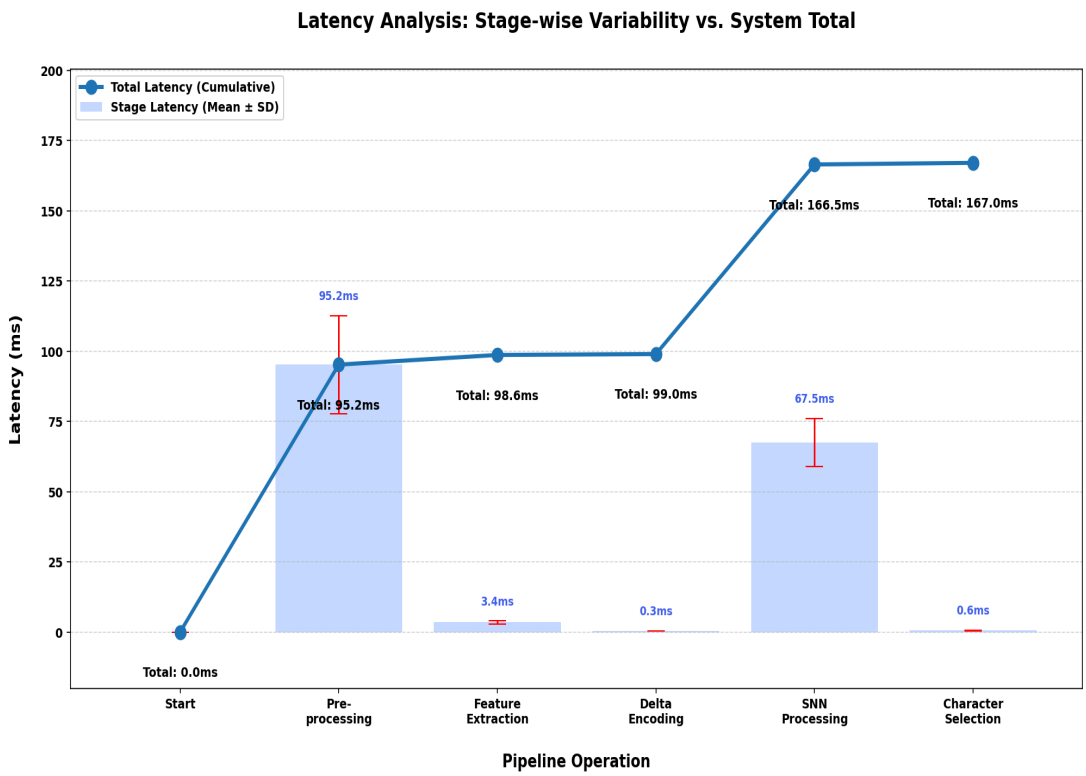


Figure 18: *Latency analysis of the full pipeline on CPU. The bar graph represents the latency for each individual component of the pipeline, while the line graph shows cumulative latency after each component's completion. The red bars indicate ± one standard deviation from the mean latency of the component.*

The energy consumption of each pipeline component on CPU was analyzed both separately and cumulatively (*Figure 19*). The total energy consumption of the system was 109.21 μJ. Similar to latency, both the preprocessing and SNN inference carried a significant load of the energy, as the preprocessing had a total energy of 65.2 μJ and the SNN inference 38.23 μJ. This is because both the preprocessing and SNN carried the most operations throughout the process, which inherently means they have the highest energy consumption. On the other hand, the feature extraction and delta encoding had very little energy consumption with the feature extraction only having 4.94 μJ, and the delta encoding having only 0.82μJ. For the character selection, the total energy consumption was 0.02 μJ which is practically negligible, given that it only makes up an extremely small portion of the total energy consumed. The energy usage of the FPGA implemented SNN was also computed, and found to be much lower than its CPU alternative, at 3.75×10^{-5}μJ.

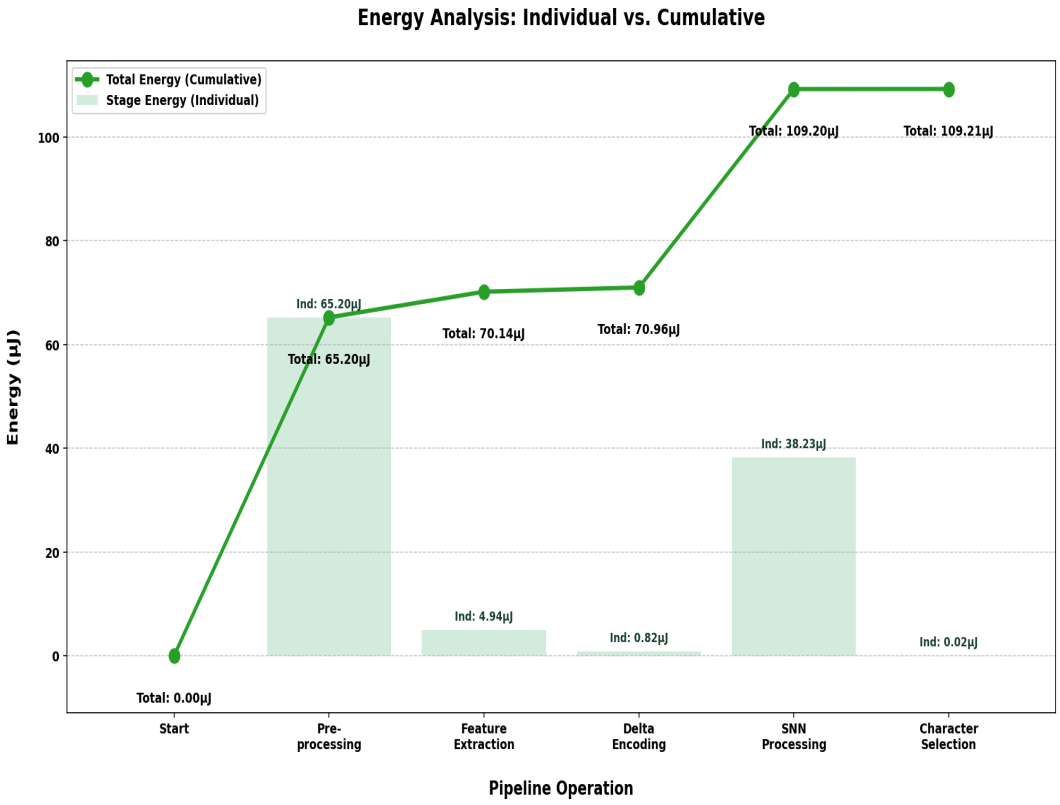


Figure 19: *Energy analysis of the full pipeline on CPU. The bar graph represents the energy consumption for each individual component of the pipeline, while the line graph shows cumulative energy consumption after each component's completion.*

The latency and energy usage of the system are summarized in *Table 7*. The components with the highest latency also carried the highest energy consumption, meaning that as the number of operations increases (energy increases), so does the latency of the system or component in question, showing a correlation between energy consumption and latency. These results indicate that improving energy efficiency in the various stages will lead not only to an overall energy save, but will also reduce the latency as well.

Table 7: *Latency and Energy of the Virtual Keyboard Pipeline for CPU and FPGA*

	Energy (µJ)		**Latency (ms)**	
	CPU	**FPGA**	**CPU**	**FPGA**
Preprocessing*	65.20	65.20	95.2 ± 17.5	95.2 ± 17.5
Feature Extraction*	4.94	4.94	3.4 ± 0.7	3.4 ± 0.7
Delta Encoding*	0.82	0.82	0.3 ± 1.0	0.3 ± 1.0
SNN Inference	38.23	3.75×10^{-4}	67.5 ± 25.3	13.7
Character Selection*	0.02	0.02	0.6 ± 2.2	0.6 ± 2.2
Total	109.21	69.98	167.0 ± 30.9	113.2 ± 30.9

*Not implemented on FPGA. Performance estimates are the same.

3.2.5.3 ANN/SNN Performance Comparison

When comparing the SNN to the ANN, the SNN carries a higher total latency than the ANN model, with the mean SNN latency being 67.50 ms while the ANN is only 5.5 ms. This is likely because the SNN processes data across multiple time steps rather than performing a single forward pass. This temporal processing will require repeated computations which increases the overall run time. On top of that, the SNN is implemented on the CPU, which does not fully take advantage of the sparse, event driven nature of the spike based computation, which further contributes to the higher latency. In comparison, the ANN performs a single, optimized forward pass, which makes it significantly faster than the SNN.

Unlike in latency however, the energy consumption for the SNN, 38.23 μJ, is less than the ANN consumption, 52.83 μJ. The lower energy consumption suggests that the spike based computation can reduce overall energy consumption even though it is processing over several steps. In contrast, the ANN, while faster, requires more energy due to its dense computation process for a single forward pass. The summary of the SNN/ANN comparison can be seen in *Table 8*. Overall, even though the ANN is faster, the SNN process in combination with FPGA acceleration offers a lower energy option that takes less than 300 ms for character inference. The FPGA energy is calculated based on the estimated power used by the board.

Table 8: *Latency and Energy Comparisons between SNN and ANN*

	SNN Inference (CPU)	SNN Inference (FPGA)	ANN Inference
Energy (μJ)	38.23	3.75×10^{-4}	52.83
Latency (ms)	67.50	13.7	5.5

It is clear from *Table 8* that using SNN Inference on FPGA hardware, is the best choice for the virtual keyboard. It has far less energy consumption, making the system more efficient. From a latency standpoint, it is not the lowest of the 3 tested configurations, but it is very close to the lower bound when looking at the entire range of the 3 cases, and it surely contributes to a pipeline that still meets the 300ms requirement.

4 Discussion

4.1 Project Successes & Key Achievements

The EEG-driven virtual keyboard system successfully met all major project requirements. One of the most important achievements was the ability to operate in pseudo-real time on CPU. The estimated latency was approximately 167.0 ms on CPU and 113.2 on FPGA, which is well below the 300 ms requirement. This confirms that the system is capable of responding quickly enough for practical user interaction.

The system also exceeded the design requirement for keyboard functionality by supporting selection from a full 36-character alphanumeric layout, rather than the minimum 4-8 keys. In addition, spike-based computation was successfully implemented in the decision-making stage through the use of a SNN. A complete working demonstration of character selection was achieved, where the system was able to process an EEG input and output the predicted characters through the full pipeline. While classification accuracy was not perfect, it was consistently above random chance, confirming that the model learned meaningful patterns in the EEG data.

The project was also successful in maintaining a low system cost. The total cost of the system was $310.57, which is well below the $600 constraint, demonstrating that an EEG-based assistive communication system can be implemented in a cost-effective manner using accessible hardware and software tools.

The overall project maintained a reproducible evaluation methodology, including structured preprocessing, consistent training and testing splits, consistent random seeding, and clearly

defined validation procedures. This ensures that results can be replicated and compared, for both academic evaluation and future development.

4.2 Problem Solving & Design Implementation

4.2.1 Preprocessing & Feature Extraction

Three preprocessing functions were developed after finalizing the preprocessing pipeline to make it more efficient and easier to use at different stages. The first function processes the full data set and generates binary labels for P300 versus non-P300 signals. The second function also processes the full dataset, but uses flash IDs instead of binary labels. The third function is a simulated real-time processing of a single character, which shows how incoming EEG data would be processed in a live system.

The original feature extraction method used extrapolated static features to represent the waveform of each signal. This design, though majorly reducing the feature space, failed to capture the temporal nature of the EEG waveforms which can be utilized in an SNN. To rectify this issue, time series information was conserved by using time downsampling and windowing to reduce feature space instead. This data format better utilized the SNN's time dependent spiking nature.

4.2.2 Spike Encoding

Initially, a deterministic rate-based encoding method was selected as the baseline approach for converting extracted EEG features into spike trains. This decision was based on early benchmarking. The rate encoding method maps normalized feature values to firing rates within a defined range, producing spike trains where higher feature values correspond to higher spike

frequencies. However, during system testing and model training, the method did not capture rapid temporal changes in the EEG signal effectively.

To address this issue, the encoding strategy was modified to use delta encoding. Unlike rate encoding, delta encoding allows the model to capture both increases and decreases in signal amplitude. This reduced the number of unnecessary spikes and improved the temporal sensitivity. After integrating delta encoding into the pipeline, training results showed a noticeable improvement in model performance compared to the original rate-based approach.

4.2.3 SNN Architecture & Training

SNN architecture design must keep in mind downstream FPGA implementation. With this in mind, we monitored the memory and LUT usage of our SNN design. Though originally our design used hidden layers of size 256 → 128 to classify signals, the volume of weights required in this layer were over the memory-limit for parallel processing on the FPGA. Because of this, we reduced our layer sizes to 128 → 64, which allowed for full utilization of the parallel processing of the FPGA.

The SNN training hyperparameters were found through many iterations of different parameter combinations. Monitoring for the highest balanced accuracy proved to be the most valuable metric to determine the success of the chosen parameters. Additionally, SNNs can be difficult to train because of sparse spiking activity, non-differentiable gradients, and temporal based dynamics [44].To ensure the spiking dynamics were still active, spike rate and membrane potential acted as diagnostic metrics. By monitoring these elements, we could determine if

learning was stagnating because of an architectural limitation or because of destruction of the spike dynamics and adjust components such as membrane potential threshold to encourage spiking output.

4.2.4 Character Selection

In order to create the character selection it was essential to understand the format of the data being input in order to code the probability matrix. This task proved to be the most challenging part when creating the process that correctly sum the total probabilities. Epochs had to be properly ordered in order for the algorithm to properly interpret the data, which was kept track of using an additional variable. Another aspect to consider was understanding how the flash matrix worked in order to figure out the highest score row and column to output the letter with the highest probability.

Various trials were run in attempts to improve the accuracy of character selection. The code that initially dealt with the interpretation of 2D arrays was converted into 1D to better deal with calculating probabilities. The original design dealt with the summing of P300 hits in the second column only. A tested revision dealt with summing of hits, non-hits, and taking the difference in order to improve the accuracy. The final version of the code extracts the probability of hits through the use of logarithmic functions.

4.2.5 FPGA Implementation

The primary challenges in the FPGA implementation involved designing a scalable neuron architecture, making sure sequential timing was correct across layers, and maintaining accurate membrane state updates for each timestep. To address this, a universal LIF-based neuron module

was developed using fixed point arithmetic, allowing consistent replication across layers (128 → 64 → 2 neurons). A structured generation function was used to scale the network while maintaining the proper function the neurons should hold. In addition, a main control module was implemented to coordinate start, done, idle, and enable signals, allowing each layer to be executed in the correct order and ensure that neuron states were updated exactly once per timestep. This prevented issues such as early accumulation or state carryover between samples. Synchronization between layers was established through specific state management, creating reliable spike propagation and stable membrane behavior throughout the network.

4.2.6 Energy & Latency

Compared to the original plan, for the project it was best determined to use FLOPs to calculate the energy consumption instead of using power draw to calculate the energy consumption. Although power draw is a more direct way of calculating energy, in this project there were several different factors that would affect the data causing inaccurate results. For instance, power draw does not take into account other operations the CPU runs besides the main pipeline. That additional usage factors into the overall energy consumption, skewing the data and resulting in inaccurate results. It would have also had to rely on onboard sensors for the FPGA board to calculate the energy consumption for the hardware, which was something to be determined as an uncertainty if it was possible. Counting FLOPs/SOPs is a more accurate process as it tracks operations through the pipeline itself, giving accurate estimates of just the system running, avoiding all other outside factors. Then, with a simple approximate conversion factor, accurate energy consumption of the pipeline can be calculated, something the power draw method simply would not have been able to accomplish.

4.3 Challenges & Limitations

The final character selection accuracy of only 48.31 ± 22.15% is not usable as a long term virtual keyboard application. If characters are only correctly selected approximately 50% of the time, there will be a lot of typos and risk of miscommunication. Though the deliverable was met as this accuracy is significantly higher than random classification, it still requires much improvement before it can reliably function as intended. This low accuracy may be because our classifier was trained on data from all subjects at once to form a general classifier. The subjects show wide variation in their P300 signal waveform (*Figure 20*). This variation could stunt the overall learning ability of the SNN, especially given the simple architecture utilized.

The accuracy may also be limited due to the architecture of the SNN. This architecture was kept shallow and simple in order for FPGA implementation to be completed in a memory and time limited environment. To try to combat this limitation, vigorous hyperparameter tuning was done; however, a more complicated or deeper network may be able to better differentiate between the signals. For example, convolutional networks are known for their ability to extract relevant information from time series data and have been used to classify P300 signals with high accuracy in the past [38]. Utilizing a convolutional spiking network could better serve to extract relevant information from the EEG data.

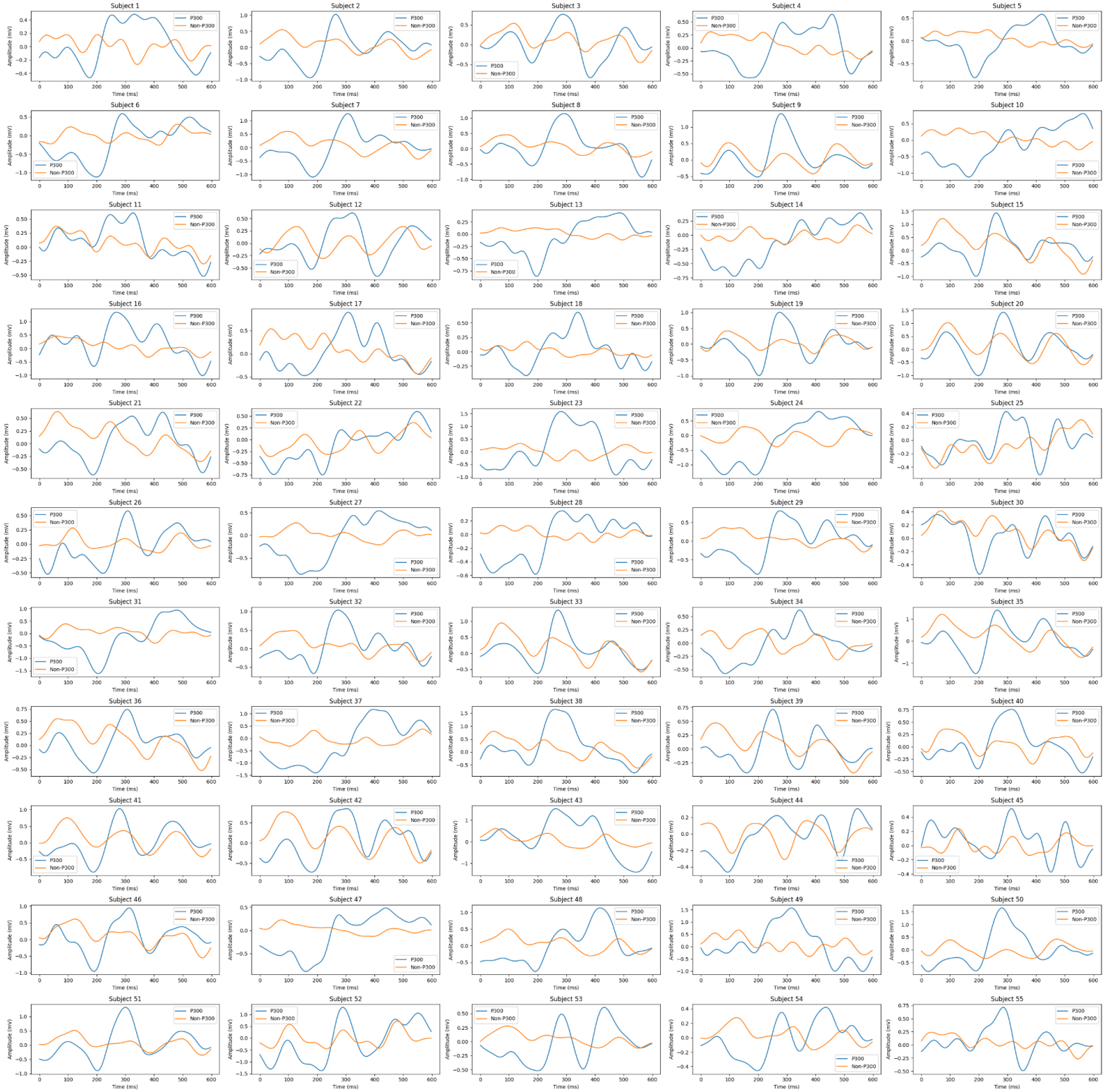


Figure 20: *Per-subject midline average P300 response (blue) vs non-P300 response (orange).*

A primary limitation of this work was the inability to achieve a full end to end FPGA application. Currently, spike encoded inputs must be manually loaded into the on chip ROM, rather than streamed. To do pseudo-real time character selection, a stable connection must be made through UART to send data between the FPGA and the external computer. While there was partial progress toward UART integration, issues encountered during ILA validation hindered full implementation. As a result, a complete end-to-end automatic pipeline could not be completed at this time. In order to design the end-to-end pipeline, a complete UART input and output integration, debugging of internal signals, and enabling reliable real time data transfer will be required.

Additionally, FPGA based classification accuracy was evaluated on only a single subject and was much lower than its CPU counterpart. The limited subject number was due to the large amount of time it would take to simulate and run subject-wise accuracy using a manual method. The low accuracy may be due to timing mismatches and differing intermediate accuracy from FPGA compared to CPU. To more efficiently generate an accuracy for each subject, implementing the UART input-output system directly into a CPU to be analyzed by Python would streamline the process compared to manually gathering the signal. To add on, incorporating the UART would allow easier accessibility and use of different sample sets to be input into the system. Further debugging of the FPGA SNN implementation's timing and tuning of its exact parameters is needed in order to improve the accuracy.

4.4 Future Work & Potential Improvements

Currently, our P300 classifier has been generally trained with multiple subjects' data. A tunable weight system could be added to the SNN which calibrates classification to a specific user. As ERPs, including P300, are widely variable between subjects [45], [46], being able to tune the classifier to a specific subject could greatly improve accuracy. Additionally, adding a confidence system would help reduce the number of repetitions required for character output. Currently, we rely on a static 15 repetitions to select the character. With a confidence system, the number of repetitions could be cut off if the confidence of the selected letter reaches a certain threshold such as 90% or 95% confidence. If we combine the subject specific calibration with confidence, the time it takes to collect the signal would be reduced.

Further improvements could be achieved by implementing the full character selection pipeline on a FPGA. Leveraging the parallel processing capabilities of FPGA hardware would enable simultaneous processing of multiple data samples, reducing system latency compared to the sequential CPU-based execution. In order to completely implement the pipeline on an FPGA, an upgraded chip with more memory availability may be needed, which would increase total development cost but decrease latency.

If additional time was provided, further adjustments and revisions could be made to the project in order to improve accuracy, latency and reduce power consumption. Although some of the reason was due to variety in the database, there could have been ways to experiment with improving character selection. Additionally, the team was unfamiliar with FPGA implementation before designing the project. With more practice and understanding, improvements to FPGA design

could be made. There is also the possibility of running the refined model using the interpretation of real EEG waveform data when hooked up to a proper device.

5 Conclusion

The project developed an EEG-driven SNN virtual keyboard for non-invasive assistive communication. The system integrates preprocessing, feature extraction, spike encoding, SNN classification, and character selection into a complete end-to-end pipeline. The system was able to identify alphanumeric characters with approximately 50% accuracy across a 36-character set. A dataset containing P300 signals from 55 different subjects triggered through random character matrix intensifications was chosen to be used for SNN training and evaluation.

Although recordings included 32 EEG channels, 8 channels with the strongest P300 responses were selected. The signals were separated into 600 ms epochs starting at the time of initial stimulus, and a baseline average from -200 ms to 0 ms was subtracted from each epoch. The signals were also filtered using a 0.5 to 15 Hz bandpass filter, went through linear detrending, and were temporally downsampled to 170 Hz. Additional time windowing from 200-400 ms from initial stimulus time was applied to isolate the signal, followed by normalization through standard scaling. To convert continuous EEG features into a form suitable for SNN processing, delta encoding was implemented using a threshold value of 0.09. This method generates spikes based on changes between consecutive time steps, allowing both increases and decreases in signal amplitude to be captured.

From there a binary classification SNN using LIF neurons was implemented, which integrated input spikes through time with a constant decay factor, and fired an output spike when a threshold was reached. For each LIF neuron, the decay factor was set to 0.95, while the threshold was set to 1. Overall SNN architecture consists of 3 fully connected layers, with the final size being 8 → 128 → 64 → 2. The sizes correspond to the 8 input channels, two hidden layer connections, and two output neurons: one for non-P300 classification and one for positive P300 classification. The SNN was trained using BPTT with surrogate gradient descent on the weighted cross entropy loss calculated during the forward pass. Character selection is implemented through an algorithm that sums logarithmic probabilities within each column and row of the speller and selects the character corresponding to the highest probability.

The FPGA implementation successfully demonstrated that a spiking neural network can be deployed in hardware, with correct spike propagation, layer synchronization, and functional neuron computation across all stages. While intermediate layers achieved high accuracy, performance degradation at the final layer and low overall accuracy indicate that further optimization is required. Further improvements should focus on refining timing alignment and system level integration to achieve more reliable performance.

The final system achieved an accuracy of 48.31 ± 22.15% for CPU implementation and a one subject accuracy of 4.76% on FPGA. Though accuracy on CPU was significantly higher than the baseline accuracy of a random weight classifier ($p << 0.5$), the FPGA accuracy was much lower. There also existed extreme variation in accuracy between subjects in the dataset. Pseudo-real time end-to-end integration of the system pipeline was implemented on software. The CPU

system achieved a total latency of 167 ± 30.9 ms with an energy consumption of 109.21 μJ, while the FPGA system achieved an estimated latency of 113.2 ± 30.9 ms and an energy consumption of 69.98 μJ. The energy consumption of both CPU and FPGA systems was less than the ANN baseline, while the latency was higher in CPU but comparable in FPGA.

This project highlights the impact in biomedical engineering of turning neural signal processing and computation into accessible and efficient assistive communication technology. Utilizing an SNN for classification proves that lower energy usage for BCI devices may be possible through this avenue. FPGA acceleration of the device was estimated to consume even less energy and be more efficient. The production of a low latency and low energy virtual keyboard will make communication more accessible and affordable to all.

References

[1] J. J. Shih, D. J. Krusienski, and J. R. Wolpaw, "Brain-Computer Interfaces in Medicine," *Mayo Clinic Proceedings*, vol. 87, no. 3, pp. 268–279, Mar. 2012, doi: 10.1016/j.mayocp.2011.12.008.

[2] A. Kawala-Sterniuk *et al.*, "Summary of over Fifty Years with Brain-Computer Interfaces—A Review," *Brain Sciences*, vol. 11, no. 1, p. 43, Jan. 2021, doi: 10.3390/brainsci11010043.

[3] Neuralink, *Translating Thought To Speech | Neuralink*, (Mar. 24, 2026). Accessed: Apr. 12, 2026. [Online Video]. Available: https://www.youtube.com/watch?v=u8mmn9lOIyw

[4] Z. Yan, J. Zhou, and W.-F. Wong, "EEG classification with spiking neural network: Smaller, better, more energy efficient," *Smart Health*, vol. 24, p. 100261, Jun. 2022, doi: 10.1016/j.smhl.2021.100261.

[5] ReHab Ibraheem Ajel, Narjis Mezaal Shati, and Firas Abdulhameed Abdullatif, "A Comprehensive Analysis of the Design of Brain-Computer Interface Systems Utilizing Electroencephalography as a Means of Measurement: A Survey," *Ingénierie des Systèmes d'Information*, vol. 29, no. 5, pp. 1967–1979, Oct. 2024, doi: 10.18280/isi.290528.

[6] L. A. Farwell and E. Donchin, "Talking off the top of your head: toward a mental prosthesis utilizing event-related brain potentials," *Electroencephalography and Clinical Neurophysiology*, vol. 70, no. 6, pp. 510–523, Dec. 1988, doi: 10.1016/0013-4694(88)90149-6.

[7] O. Maslova *et al.*, "Non-invasive EEG-based BCI spellers from the beginning to today: a mini-review," *Front. Hum. Neurosci.*, vol. 17, Aug. 2023, doi: 10.3389/fnhum.2023.1216648.

[8] K. Won, M. Kwon, M. Ahn, and S. C. Jun, “EEG Dataset for RSVP and P300 Speller Brain-Computer Interfaces,” *Sci Data*, vol. 9, no. 1, p. 388, Jul. 2022, doi: 10.1038/s41597-022-01509-w.

[9] T. W. Picton, “The P300 Wave of the Human Event-Related Potential,” *Journal of Clinical Neurophysiology*, vol. 9, no. 4, p. 456, Oct. 1992.

[10] J. del R. Millán *et al.*, “Combining Brain–Computer Interfaces and Assistive Technologies: State-of-the-Art and Challenges,” *Front. Neurosci.*, vol. 4, Sep. 2010, doi: 10.3389/fnins.2010.00161.

[11] B. Ayasi, C. Carmona, M. Saleh, and A. García-Vico, “A Practical Tutorial on Spiking Neural Networks: Comprehensive Review, Models, Experiments, Software Tools, and Implementation Guidelines,” *Eng*, vol. 6, p. 304, Nov. 2025, doi: 10.3390/eng6110304.

[12] A. Rosado-Muñoz, M. Bataller-Mompeán, and J. Guerrero-Martínez, “FPGA implementation of Spiking Neural Networks,” *IFAC Proceedings Volumes*, vol. 45, no. 4, pp. 139–144, Jan. 2012, doi: 10.3182/20120403-3-DE-3010.00074.

[13] F. Liu *et al.*, “Exploiting Temporal-Unrolled Parallelism for Energy-Efficient SNN Acceleration,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 10, pp. 1749–1764, Oct. 2024, doi: 10.1109/TPDS.2024.3415712.

[14] F. Yan, A. Koch, and O. Sinnen, “A survey on FPGA-based accelerator for ML models,” Dec. 20, 2024, *arXiv*: arXiv:2412.15666. doi: 10.48550/arXiv.2412.15666.

[15] S. Charitos, “Spiking Neural Networks and FPGAs at the deep edge? The mosquito flies fast! - EdgeAI.” Accessed: Feb. 12, 2026. [Online]. Available: https://edge-ai-tech.eu/spiking-neural-networks-and-fpgas-at-the-deep-edge-the-mosquito-flies-fast/

[16] J. Fischer, T. Milekovic, G. Schneider, and C. Mehring, “Low-latency multi-threaded processing of neuronal signals for brain-computer interfaces,” *Front. Neuroeng.*, vol. 7, Jan. 2014, doi: 10.3389/fneng.2014.00001.

[17] IEEE, “IEEE Code of Ethics | IEEE,” Ieee.org. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8

[18] FDA, “Software as a Medical Device (SaMD),” FDA. [Online]. Available: https://www.fda.gov/medical-devices/digital-health-center-excellence/software-medical-device-samd

[19] International Organization for Standardization, “ISO 14971:2019,” ISO.

[20] C. Conrad and C. Heggie, “Legal and Ethical Challenges Raised By Advances in Brain-Computer Interface Technology,” *Canadian Journal of Law and Technology*, vol. 21, no. 2, p. 201, Mar. 2024.

[21] “Nexys A7 AMD Artix™ 7 FPGA Trainer Board: Recommended for ECE Curriculum,” Digilent. Accessed: Feb. 03, 2026. [Online]. Available: https://digilent.com/shop/nexys-a7-amd-artix-7-fpga-trainer-board-recommended-for-ece-curriculum/

[22] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[23] P. Virtanen *et al.*, “SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nat Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020, doi: 10.1038/s41592-019-0686-2.

[24] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Dec. 03, 2019, *arXiv*: arXiv:1912.01703. doi: 10.48550/arXiv.1912.01703.

[25] J. K. Eshraghian *et al.*, “Training spiking neural networks using lessons from deep learning,” *Proceedings of the IEEE*, vol. 111, pp. 1016–1054, 2023.

[26] J. Polich and K. L. Herbst, “P300 as a clinical assay: rationale, evaluation, and findings,” *International Journal of Psychophysiology*, vol. 38, no. 1, pp. 3–19, Oct. 2000, doi: 10.1016/S0167-8760(00)00127-6.

[27] J. Pan, X. Chen, N. Ban, J. He, J. Chen, and H. Huang, “Advances in P300 brain–computer interface spellers: toward paradigm design and performance evaluation,” *Front. Hum. Neurosci.*, vol. 16, Dec. 2022, doi: 10.3389/fnhum.2022.1077717.

[28] D. J. Krusienski, E. W. Sellers, D. J. McFarland, T. M. Vaughan, and J. R. Wolpaw, “Toward enhanced P300 speller performance,” *Journal of Neuroscience Methods*, vol. 167, no. 1, pp. 15–21, Jan. 2008, doi: 10.1016/j.jneumeth.2007.07.017.

[29] S. K. R. Singanamalla and C.-T. Lin, “Spike-Representation of EEG Signals for Performance Enhancement of Brain-Computer Interfaces,” *Front. Neurosci.*, vol. 16, Apr. 2022, doi: 10.3389/fnins.2022.792318.

[30] D. Auge, J. Hille, E. Mueller, and A. Knoll, “A Survey of Encoding Techniques for Signal Processing in Spiking Neural Networks,” *Neural Process Lett*, vol. 53, no. 6, pp. 4693–4710, Dec. 2021, doi: 10.1007/s11063-021-10562-2.

[31] K. Wang, X. Hao, J. Wang, and B. Deng, “Comparison and Selection of Spike Encoding Algorithms for SNN on FPGA,” *IEEE Transactions on Biomedical Circuits and Systems*, vol. 17, no. 1, pp. 129–141, Feb. 2023, doi: 10.1109/TBCAS.2023.3238165.

[32] P. Goel, H. Liu, D. Brown, and A. Datta, “Spiking Neural Network Based Classification of Task-Evoked EEG Signals,” in *Proceedings of the 10th international conference on*

Knowledge-Based Intelligent Information and Engineering Systems - Volume Part I, Nov. 2006, pp. 825–832. doi: 10.1007/11892960_99.

[33] J. Han, Z. Li, W. Zheng, and Y. Zhang, "Hardware implementation of spiking neural networks on FPGA," *Tsinghua Science and Technology*, vol. 25, no. 4, pp. 479–486, Aug. 2020, doi: 10.26599/TST.2019.9010019.

[34] A. H. Ali, M. Navardi, and T. Mohsenin, "Energy-Aware FPGA Implementation of Spiking Neural Network with LIF Neurons," Nov. 03, 2024, *arXiv*: arXiv:2411.01628. doi: 10.48550/arXiv.2411.01628.

[35] A. Carpegna, A. Savino, and S. D. Carlo, "Spiker+: a framework for the generation of efficient Spiking Neural Networks FPGA accelerators for inference at the edge," *IEEE Trans. Emerg. Topics Comput.*, vol. 13, no. 3, pp. 784–798, Jul. 2025, doi: 10.1109/TETC.2024.3511676.

[36] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," Feb. 06, 2015, *arXiv*: arXiv:1502.01852. doi: 10.48550/arXiv.1502.01852.

[37] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," Jan. 30, 2017, *arXiv*: arXiv:1412.6980. doi: 10.48550/arXiv.1412.6980.

[38] H. Shan, Y. Liu, and T. Stefanov, "A Simple Convolutional Neural Network for Accurate P300 Detection and Character Spelling in Brain Computer Interface," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18*, International Joint Conferences on Artificial Intelligence Organization, Jul. 2018, pp. 1604–1610. doi: 10.24963/ijcai.2018/222.

[39] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

[40] T. Vatwani, A. Dutt, D. Bhattacharjee, and A. Chattopadhyay, “Floating Point Multiplication Mapping on ReRAM Based In-memory Computing Architecture,” in *2018 31st International Conference on VLSI Design and 2018 17th International Conference on Embedded Systems (VLSID)*, Pune: IEEE, Jan. 2018, pp. 439–444. doi: 10.1109/VLSID.2018.104.

[41] M. Sorbaro, Q. Liu, M. Bortone, and S. Sheik, “Optimizing the Energy Consumption of Spiking Neural Networks for Neuromorphic Applications,” *Front Neurosci*, vol. 14, p. 662, Jun. 2020, doi: 10.3389/fnins.2020.00662.

[42] A. K. Jain, J. Mao, and K. M. Mohiuddin, “Artificial neural networks: a tutorial,” *Computer*, vol. 29, no. 3, pp. 31–44, Mar. 1996, doi: 10.1109/2.485891.

[43] A. Nandi and H. Ahmed, “Artificial Neural Networks (ANNs),” in *Condition Monitoring with Vibration Signals*, 2019, pp. 239–258. doi: 10.1002/9781119544678.ch12.

[44] Z. Rudnicka, J. Szczepanski, and A. Pregowska, “Accuracy-Efficiency Trade-Offs in Spiking Neural Networks: A Lempel-Ziv Complexity Perspective on Learning Rules,” Mar. 01, 2026, *arXiv*: arXiv:2506.06750. doi: 10.48550/arXiv.2506.06750.

[45] F. Li *et al.*, “Inter-subject P300 variability relates to the efficiency of brain networks reconfigured from resting- to task-state: Evidence from a simultaneous event-related EEG-fMRI study,” *NeuroImage*, vol. 205, p. 116285, Jan. 2020, doi: 10.1016/j.neuroimage.2019.116285.

[46] A. Melnik *et al.*, “Systems, Subjects, Sessions: To What Extent Do These Factors Influence EEG Data?,” *Front Hum Neurosci*, vol. 11, p. 150, Mar. 2017, doi: 10.3389/fnhum.2017.00150.

Appendix

EEG-Driven Spiking Neural Network Virtual Keyboard
User Manual

---

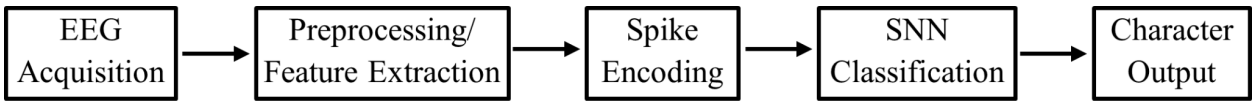
1. System Overview

This system is a brain-computer interface (BCI) that enables a user to select characters using brain activity measured through electroencephalography (EEG). The system detects neural responses associated with attention (specifically the P300 event-related potential) and converts these signals into character outputs using a trained spiking neural network (SNN).

The system is designed to operate on either:

- ➢ software (CPU/GPU environment) for development and testing, or
- ➢ hardware (FPGA implementation) for low-power, real-time deployment.

The complete processing pipeline includes:



2. System Setup & Physical Configuration

2.1 User Setup (EEG Acquisition)

To operate the system with a real user, the following setup is required [1]:

- ➢ A 32-channel EEG cap placed according to the international 10–20 system
- ➢ Conductive gel applied to ensure low-impedance electrode contact
- ➢ EEG acquisition hardware (e.g., BioSemi ActiveTwo system)
- ➢ A display screen positioned approximately 1 meter in front of the user

The user should:

- ➢ Sit comfortably and remain still
- ➢ Minimize eye and muscle movement
- ➢ Focus attention on specific characters presented on the screen

During operation, EEG signals are continuously recorded at approximately 512 Hz sampling rate [1].

2.2 P300 Speller Interface

The system uses a 6×6 character matrix, containing:

- ➢ Letters A–Z
- ➢ Numbers 1–9
- ➢ Space character

	Col 1 Flash 7	Col 2 Flash 8	Col 3 Flash 9	Col 4 Flash 10	Col 5 Flash 11	Col 6 Flash 12
Row 1/Flash 1	A	B	C	D	E	F
Row 2/Flash 2	G	H	I	J	K	L
Row 3/Flash 3	M	N	O	P	Q	R
Row 4/Flash 4	S	T	U	V	W	X
Row 5/Flash 5	Y	Z	1	2	3	4
Row 6/Flash 6	5	6	7	8	9	_

Characters are selected using a flashing paradigm:

- ➢ Rows and columns flash in random order
- ➢ The user focuses on a target character
- ➢ When the correct row or column flashes, a P300 response (~300 ms) is generated in the EEG signal
- ➢ The series of flashes is repeated 15 times for increased accuracy

The selected character is determined from the intersection of the detected row and column responses.

3. Data Acquisition & Training Source

This system is trained using a publicly available EEG dataset. 55 participants performed RSVP and P300 speller tasks, with EEG recorded using 32 electrodes at 512 Hz [1]. Participants completed both calibration runs (training data) and test runs (evaluation data).

Data includes:

- ➢ Raw EEG signals
- ➢ Target vs. non-target labels
- ➢ Stimulus timing markers
- ➢ Character sequences

4. Signal Processing & Pipeline Model

Once EEG data is acquired (or loaded from dataset), the system processes it as follows:

4.1 Preprocessing

- ➢ Bandpass filtering (15 Hz)
- ➢ Baseline correction
- ➢ Channel selection
- ➢ Noise/artifact reduction

4.2 Feature Extraction

- ➢ Epoch segmentation (200-400 ms after stimulus)
- ➢ Temporal downsampling

4.3 Spike Encoding

- ➢ Continuous EEG features are converted into spike trains
- ➢ Delta encoding generates spikes when signal changes exceed a threshold of 0.09

4.4 SNN Classification

- ➢ The encoded spike trains are input into a spiking neural network (SNN) which processes temporal spike patterns across channels and time
- ➢ The model performs binary classification, both target (P300 response detected) and non-target (no significant response)
- ➢ Output is generated for each stimulus event (row or column flash)

4.5 Character Selection

- ➢ Each stimulus corresponds to either a row or a column in the 6×6 matrix
- ➢ The system determines the row and column with the highest target probability
- ➢ The final character is selected at the intersection of the identified row and column

5. System Operation

5.1 Software (CPU/GPU)

The software implementation is used for development, testing, and evaluation using pre-recorded EEG datasets.

Step 1: Download and Install Dependencies

- ➢ Make sure python is installed on your computer

  To install python, visit their website https://www.python.org/downloads/ and follow the instructions for download

- Download the git repository here: https://github.com/MacFadyenK/VirtualKeyboard
- Create a virtual environment and activate it. Navigate to the directory containing the repository in the command window then type:

  Windows: python -m venv .venv
  .venv\Scripts\activate

  MacOS: python3 -m venv .venv
  source .venv/bins/activate
- Install dependencies:

  pip install -r requirements.txt

Step 2: Launch Program

- Run the appropriate script:

  CPU: python *VirtualKeyboard_cmd_ln_cpu.py*

Step 3: Select Dataset

- Enter a subject number between 1–55. The system loads the corresponding EEG dataset file.
- *Program Example:*

```
Enter subject number (1-55): 10
```

Step 4: Select Target Character

- Enter a character (A–Z) for processing. The system extracts all EEG epochs associated with that character.
- *Program Example:*

```
Enter character to select (A-Z): B
```

Step 5: Automated Processing

- The system performs the full pipeline automatically.

Step 6: Output Display

- The system displays the predicted character (selected output) and, for validation, the correct character (correct output).
- *Program Example:*

```
Selected output: B
Correct output: B
```

Step 7: Continue, Clear, or Exit

- Repeat character selection (Step 3) as needed

- ➢ Enter CLEAR to clear all previously recorded outputs
- ➢ Enter QUIT to terminate the program

5.2 Hardware (FPGA)

The prospective hardware implementation enables real-time, low-power operation of the system. (future implementation)

Step 1: System Setup

- ➢ Connect EEG acquisition hardware to the processing unit
- ➢ Connect FPGA board via USB or appropriate interface
- ➢ Run the appropriate script:

  FPGA: python *VirtualKeyboard_cmd_ln_fpga.py*
- ➢ Ensure all required firmware and bitstreams are loaded

Step 2: User Preparation

- ➢ Fit EEG cap and ensure proper electrode contact
- ➢ Position the user in front of the display
- ➢ Instruct the user to remain still and focus on target characters

Step 3: Signal Acquisition

- ➢ EEG signals are continuously recorded from all channels
- ➢ Data is streamed directly to the processing system

Step 4: Real-Time Processing

- ➢ The system executes the pipeline in real time with SNN classification on the FPGA.

Step 5: Character Output

- ➢ The system identifies the most likely row and column
- ➢ The selected character is displayed on the screen

Step 6: Continuous Operation

- ➢ The process repeats for each character selection
- ➢ Output text is built sequentially as the user continues

6. Performance Metrics

The system performance is evaluated in terms of latency and energy consumption for each stage of the processing pipeline, as well as the full end-to-end system. Latency is defined as the time

required to process data at each stage, while energy consumption represents the computational cost associated with each operation.

Pipeline Stage	Software (CPU/GPU)		Hardware (FPGA)	
	Latency (ms)	Energy Consumption (µJ)	Latency (ms)	Energy Consumption (µJ)
Preprocessing	95.2 ± 17.5	65.20	95.2 ± 17.5	65.2
Feature Extraction	3.4 ± 0.7	4.94	3.4 ± 0.7	4.94
Spike Encoding	0.3 ± 1.0	0.82	0.3 ± 1.0	0.82
SNN Classification	67.5 ± 25.3	38.23	13.7	3.75 x 10-4
Character Selection	0.6 ± 2.2	0.02	0.6 ± 2.2	0.02
Full Pipeline Total:	**167.0 ± 30.9**	**109.21**	**113.2 ± 30.9**	**69.98**

7. Limitations & Constraints

- ➢ Current system uses pre-recorded EEG data
- ➢ Performance varies across subjects
- ➢ Noise and artifacts can affect accuracy
- ➢ Real-time deployment requires optimized hardware implementation in future updates

8. Safety & User Considerations

- ➢ EEG is non-invasive and safe
- ➢ Users should avoid excessive movement during operation
- ➢ Sessions should be limited to avoid fatigue and reduced attention

9. References

[1] K. Won, M. Kwon, M. Ahn, and S. C. Jun, "EEG Dataset for RSVP and P300 Speller Brain-Computer Interfaces," *Scientific Data*, vol. 9, no. 1, p. 388, Jul. 2022, doi: https://doi.org/10.1038/s41597-022-01509-w.